TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn : Công nghệ thông tin

BÀI TẬP LỚN

MÔN HỌC

TRÍ TUỆ NHÂN TẠO & HỌC MÁY

ĐỀ TÀI : HỆ THỐNG ĐIỂM DANH BẰNG NHẬN DẠNG MẶT NGƯỜI

Sinh viên : Vũ Lân

Lớp : K58KTP

Giáo viên hướng dẫn : TS. Nguyễn Tuấn Linh

Thái nguyên-2026

TRƯỜNG ĐHKTCN	**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM**
KHOA ĐIỆN TỬ	***Độc lập - Tự do - Hạnh phúc***

BÀI TẬP LỚN

MÔN HỌC TRÍ TUỆ NHÂN TẠO

BỘ MÔN CÔNG NGHỆ THÔNG TIN

Sinh viên: Vũ Lân

Lớp: K58KTP Ngành:Kỹ thuật phần mềm

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề: 13/12/2025 Ngày hoàn thành: 11/3/2026

Tên đề tài. Hệ thống điểm danh bằng nhận dạng mặt người

Yêu cầu :

Mô tả bài toán :

- Xây dựng hệ thống điểm danh tự động dựa trên công nghệ nhận dạng khuôn mặt.
- Hệ thống sử dụng camera (webcam) để thu nhận hình ảnh khuôn mặt người dùng.
- Cho phép đăng ký khuôn mặt của sinh viên/người dùng trước khi điểm danh.
- Thực hiện nhận dạng khuôn mặt theo thời gian thực và xác định danh tính.
- Ghi nhận kết quả điểm danh (tên người dùng, thời gian) và lưu trữ vào file hoặc cơ sở dữ liệu.
- Hệ thống hoạt động ổn định, dễ sử dụng và có khả năng mở rộng.

Mô tả dữ liệu:

_Dữ liệu đầu vào: +Hình ảnh khuôn mặt được thu từ webcam.

+Tập ảnh khuôn mặt đã đăng ký trước, mỗi ảnh tương ứng với một người dùng.

_Dữ liệu lưu trữ: +Ảnh khuôn mặt được lưu trong thư mục face_data.

+File kết quả điểm danh (attendance.csv) bao gồm:

-Tên người dùng

-Thời gian điểm danh

_Định dạng dữ liệu: +Ảnh: .jpg, .png

+File điểm danh: .csv

Mô hình học máy:

_Hệ thống sử dụng mô hình nhận dạng khuôn mặt dựa trên phương pháp trích xuất đặc trưng khuôn mặt.
Cụ thể:

- Sử dụng thuật toán Face Recognition để mã hóa khuôn mặt thành vector đặc trưng (128 chiều).
- Áp dụng phương pháp so khớp khoảng cách (Euclidean distance) để so sánh khuôn mặt đầu vào với các khuôn mặt đã đăng ký.
- Nếu khoảng cách nhỏ hơn ngưỡng cho phép, khuôn mặt được xem là trùng khớp và hệ thống xác nhận danh tính.

_Phù hợp với hệ thống điểm danh thời gian thực.

Giao diện:

- Giao diện hiển thị video trực tiếp từ webcam.
- Khuôn mặt được phát hiện sẽ được vẽ khung chữ nhật bao quanh.
- Tên người dùng được hiển thị ngay trên khung khuôn mặt khi nhận dạng thành công.
- Hệ thống tự động ghi nhận điểm danh và thông báo trạng thái trong cửa sổ chương trình hoặc trên terminal.
- Giao diện đơn giản, dễ sử dụng, phù hợp cho môi trường lớp học và văn phòng.

GIÁO VIÊN HƯỚNG DẪN
(ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày….tháng…..năm 2025

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

MỤC LỤC

LỜI NÓI ĐẦU 7
DANH MỤC HÌNH ẢNH 8
DANH MỤC BẢNG BIỂU 9
CHƯƠNG 1 : TỔNG QUAN 10
1.1 Đặt vấn đề (Problem Statement) 10
1.2. Lý do chọn đề tài (Motivation) 10
1.3. Mục tiêu của đề tài (Objectives) 11
1.4. Đóng góp của báo cáo 11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT & CÁC NGHIÊN CỨU LIÊN QUAN 12
2.1. Tổng quan về nhận dạng khuôn mặt 12
2.2. Các phương pháp nhận dạng khuôn mặt phổ biến 12
2.2.1. Haar Cascade (Viola-Jones) 13
2.2.2. HOG + SVM (Histogram of Oriented Gradients + Support Vector Machine) 13
2.2.3. CNN-based (Convolutional Neural Network) – FaceNet, ArcFace, dlib ResNet-34 14
2.3. Các công nghệ và thư viện sử dụng 14
2.3.1. Python 3.11 15
2.3.2. face_recognition (dựa trên dlib) 15
2.3.3. OpenCV (cv2) 16
2.3.4. Flask 16
2.3.5. SQL Server (Microsoft SQL Server Express) 17
2.3.6. Pillow (PIL) 17
2.3.7. pyodbc 17
2.3.8. Các thư viện hỗ trợ khác 17
2.3.9. Tổng hợp cách các công nghệ kết hợp 18
CHƯƠNG 3 : PHÂN TÍCH PHƯƠNG PHÁP THỰC HIỆN & THIẾT KẾ HỆ THỐNG 20

3.1. Mô tả dữ liệu (Data Description) 20
3.1.1. Nguồn dữ liệu 20
3.2. Khám phá và Tiền xử lý dữ liệu (EDA & Preprocessing) 21
3.3.1. Mô tả kiến trúc 22
3.3.2. Hàm mất mát và thuật toán tối ưu 23
3.3.3. Yêu cầu phi chức năng 23
3.4. Sơ đồ Use-case 24
3.5. Sơ đồ tuần tự 25
3.6. Sơ đồ luồng dữ liệu 27
3.7. Thiết kế cơ sở dữ liệu 28
3.7.1. Lý do lựa chọn SQL Server 28
3.7.2. Cấu trúc các bảng chính 28
3.7.3. Mối quan hệ giữa các bảng 29
3.8. Thiết kế thuật toán chính 30
3.8.1. Thuật toán đăng ký khuôn mặt 30
3.8.2. Thuật toán điểm danh thời gian thực 32
3.8.3. Cơ chế throttle chống spam (phân tích chi tiết) 34
CHƯƠNG 4 : THỰC HIỆN VÀ KẾT QUẢ THỰC NGHIỆM 36
4.1. Môi trường triển khai 36
4.2. Các siêu tham số (Hyperparameters) 37
4.3. Tiêu chí đánh giá (Evaluation Metrics) 37
4.4. Các file chính và chức năng 38
4.5. Kết quả thực nghiệm 40
4.5.1. Đăng ký khuôn mặt 41
4.5.2. Điểm danh thời gian thực 43
4.5.3. Trang Admin Dashboard 45
4.5.4. Bảng kết quả tổng hợp 46
5.2. Đánh giá hiệu năng 47

5.3. Phân tích lỗi (Error Analysis) 48
5.3.1. False Positive (Nhận nhầm người khác) 48
5.3.2. False Negative (Không nhận ra người dùng hợp lệ) 48
5.3.3. Lỗi khác 49
5.4. Hạn chế của hệ thống hiện tại 49
CHƯƠNG 6 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN (CONCLUSION) 51
6.1. Những bài học kinh nghiệm rút ra 51
6.2. Hướng phát triển trong tương lai 51
TÀI LIỆU THAM KHẢO 53

LỜI NÓI ĐẦU

Trong thời đại công nghệ số 4.0, việc áp dụng trí tuệ nhân tạo vào các hoạt động quản lý là xu hướng tất yếu. Việc áp dụng các công nghệ thông minh không chỉ giúp nâng cao hiệu quả làm việc mà còn góp phần giảm thiểu sai sót do yếu tố con người gây ra. Điểm danh truyền thống bằng cách gọi tên hoặc ký tay vẫn tồn tại nhiều hạn chế : tốn thời gian, dễ gian lận, thiếu chính xác.

Đề tài "Hệ thống điểm danh bằng nhận dạng khuôn mặt" được em thực hiện nhằm xây dựng một hệ thống tự động, nhanh chóng, chính xác và có thể triển khai thực tế trong lớp học, văn phòng. Đề tài hướng tới việc ứng dụng công nghệ trí tuệ nhân tạo và xử lý ảnh để xây dựng một hệ thống điểm danh tự động, có khả năng nhận dạng khuôn mặt người dùng theo thời gian thực, từ đó ghi nhận thông tin điểm danh một cách nhanh chóng và chính xác.

Em xin chân thành cảm ơn TS. Nguyễn Tuấn Linh đã hướng dẫn tận tình trong quá trình thực hiện đề tài.

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ tổng quan công nghệ 18
Hình 3.1: Sơ đồ tổng quát 24
Hình 3.2: sơ đồ tuần tự đăng nhập 25
Hình 3.3: Điểm danh tự động 25
Hình 3.4: Đăng ký khuôn mặt 26
Hình 3.5: Quản lý người dùng (Admin thêm/xóa người dùng) 26
Hình 3.6: Xem danh sách điểm danh (Admin xem lịch sử) 27
Hình 3.7: Sơ đồ luồng dữ liệu 27
Hình 3.8: Sơ đồ quan hệ thực thể (ERD) của cơ sở dữ liệu 30
Hình 4.1: Cấu trúc thư mục dự án 40
Hình 4.2: kết quả điểm danh 40
Hình 4.3: Trang đăng ký khuôn mặt từ Upload ảnh 41
Hình 4.4: Trang đăng ký khuôn mặt – tab chụp webcam 42
Hình 4.5: Console Flask khi đăng ký thành công. 42
Hình 4.6: Stream webcam ngoài (index 1) với hộp xanh + tên hiển thị. 43
Hình 4.7: Console Flask khi điểm danh nhiều lần (hiển thị log [ATTENDANCE]) 43
Hình 4.8: Bảng Attendance trong SQL Server (nhiều bản ghi cùng UserId, thời gian khác nhau) 44
Hình 4.9: Trang Admin Dashboard – bảng điểm danh hôm nay. 45
Hình 4.10: Thông báo "Xóa thành công" sau khi xóa người dùng. 46

DANH MỤC BẢNG BIỂU

Bảng 2.1: Tóm tắt công nghệ.....18

Bảng 3.1: Users – Lưu thông tin người dùng và vector đặc trưng khuôn mặt 28

Bảng 3.2: Attendance – Lưu lịch sử điểm danh 29

Bảng 4.1: Kết quả đánh giá hiệu năng.....46

CHƯƠNG 1 : TỔNG QUAN

1.1 Đặt vấn đề (Problem Statement)

Trong môi trường giáo dục và làm việc hiện nay, công tác điểm danh vẫn chủ yếu được thực hiện theo phương pháp thủ công như gọi tên hoặc ký xác nhận. Cách làm này tồn tại nhiều hạn chế như tốn thời gian, dễ xảy ra gian lận, thiếu tính chính xác và khó quản lý khi số lượng người tham gia lớn.

Bài toán mà em giải quyết trong đề tài này là **xây dựng một hệ thống điểm danh tự động dựa trên công nghệ nhận dạng khuôn mặt**, nhằm thay thế phương pháp điểm danh truyền thống, giúp quá trình diễn ra nhanh chóng, chính xác và tiện lợi hơn.

- **Input của hệ thống**:
  Hình ảnh khuôn mặt được thu nhận trực tiếp từ webcam và tập ảnh khuôn mặt đã được đăng ký trước của người dùng.
- **Output của hệ thống**:
  Thông tin điểm danh bao gồm tên người dùng và thời gian điểm danh, được lưu trữ vào file dữ liệu (attendance.csv).

1.2. Lý do chọn đề tài (Motivation)

Nhận dạng khuôn mặt là một trong những lĩnh vực ứng dụng quan trọng và phổ biến của trí tuệ nhân tạo trong đời sống hiện nay, đặc biệt trong các hệ thống giám sát, xác thực danh tính và quản lý con người.

Em chọn đề tài này vì:

- Bài toán **có tính thực tiễn cao**, có thể áp dụng trực tiếp trong lớp học, văn phòng, cơ quan và nhiều doanh nghiệp hiện nay.
- Góp phần **giảm thời gian và công sức** cho công tác điểm danh.
- Hạn chế tình trạng **điểm danh hộ**, nâng cao tính trung thực và chính xác.
- Là cơ hội để em **vận dụng kiến thức trí tuệ nhân tạo** đã học vào một bài toán cụ thể, gắn liền với thực tế.

Ngoài ra, đề tài giúp em tiếp cận và làm quen với các kỹ thuật xử lý ảnh, học máy và nhận dạng khuôn mặt – những lĩnh vực quan trọng trong ngành Kỹ thuật phần mềm hiện nay.

1.3. Mục tiêu của đề tài (Objectives)

Mục tiêu của đề tài là xây dựng một hệ thống điểm danh tự động bằng nhận dạng khuôn mặt với các yêu cầu cụ thể sau:

- Thu nhận hình ảnh khuôn mặt người dùng từ webcam.
- Đăng ký và lưu trữ dữ liệu khuôn mặt của người dùng.
- Nhận dạng khuôn mặt theo thời gian thực.
- Xác định đúng danh tính người dùng với độ chính xác cao.
- Ghi nhận và lưu trữ kết quả điểm danh (tên, thời gian).
- Hệ thống hoạt động ổn định, dễ sử dụng và có khả năng mở rộng.

1.4. Đóng góp của báo cáo

Trong báo cáo này, em đã thực hiện và đạt được các nội dung sau:

- Phân tích bài toán điểm danh tự động bằng nhận dạng khuôn mặt.
- Thiết kế hệ thống điểm danh dựa trên công nghệ nhận dạng khuôn mặt.
- Xây dựng mô hình nhận dạng khuôn mặt sử dụng phương pháp trích xuất đặc trưng và so khớp khoảng cách.
- Cài đặt chương trình điểm danh hoạt động theo thời gian thực.
- Lưu trữ và quản lý dữ liệu điểm danh một cách rõ ràng, dễ kiểm tra.

Báo cáo thể hiện quá trình áp dụng kiến thức trí tuệ nhân tạo vào một bài toán thực tế, góp phần nâng cao hiểu biết và kỹ năng của em trong lĩnh vực này.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT & CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Tổng quan về nhận dạng khuôn mặt

Nhận dạng khuôn mặt (Face Recognition) là một lĩnh vực quan trọng của trí tuệ nhân tạo và thị giác máy tính, tập trung vào việc xác định hoặc xác thực danh tính con người dựa trên các đặc trưng sinh trắc học của khuôn mặt. Đây là một trong những phương pháp nhận dạng sinh trắc học phổ biến nhất hiện nay do tính tự nhiên, không xâm lấn và thuận tiện trong quá trình sử dụng.

Một hệ thống nhận dạng khuôn mặt hoàn chỉnh thường bao gồm ba giai đoạn chính:

- Phát hiện khuôn mặt (Face Detection): Xác định vị trí khuôn mặt trong ảnh hoặc video.
- Trích xuất đặc trưng (Feature Extraction): Chuyển đổi khuôn mặt thành một tập đặc trưng số (vector đặc trưng).
- Nhận dạng hoặc xác thực (Recognition/Verification): So sánh đặc trưng khuôn mặt đầu vào với dữ liệu đã lưu để xác định danh tính.

Trong thực tế, nhận dạng khuôn mặt được ứng dụng rộng rãi trong nhiều lĩnh vực như:

- Hệ thống điểm danh tự động trong giáo dục.
- Kiểm soát ra vào, an ninh giám sát.
- Xác thực người dùng trong thiết bị di động.
- Hệ thống thanh toán và định danh điện tử.

Đối với bài toán điểm danh, nhận dạng khuôn mặt giúp loại bỏ các phương pháp thủ công truyền thống như gọi tên hoặc ký xác nhận, từ đó nâng cao tính chính xác, giảm gian lận và tiết kiệm thời gian.

2.2. Các phương pháp nhận dạng khuôn mặt phổ biến

Nhận dạng khuôn mặt là một bài toán sinh trắc học (biometric) quan trọng trong trí tuệ nhân tạo. Hiện nay có nhiều phương pháp được sử dụng, từ cổ điển đến hiện đại.

Dưới đây là phân tích ba phương pháp phổ biến nhất, bao gồm ưu nhược điểm và lý do lựa chọn phương pháp trong đề tài.

2.2.1. Haar Cascade (Viola-Jones)

- **Tác dụng**: Phát hiện khuôn mặt nhanh chóng bằng cách sử dụng các đặc trưng Haar-like (hình chữ nhật đen-trắng) kết hợp với bộ phân loại AdaBoost.
- **Ưu điểm**:

  + Tốc độ rất nhanh (chạy được trên CPU yếu, thậm chí thiết bị nhúng).

  + Không cần huấn luyện phức tạp.

- **Nhược điểm**:

  + Độ chính xác thấp, đặc biệt khi khuôn mặt nghiêng, ánh sáng kém, đeo kính hoặc khẩu trang.

  + Dễ phát hiện sai (false positive cao).

- **Ứng dụng**: Chủ yếu dùng để phát hiện khuôn mặt ban đầu (face detection), không dùng cho nhận dạng (face recognition).

2.2.2. HOG + SVM (Histogram of Oriented Gradients + Support Vector Machine)

- **Tác dụng**: Đây là phương pháp được dlib sử dụng trong các phiên bản cũ.

  + Trích xuất đặc trưng HOG (mô tả hướng gradient của ảnh).

  + Sử dụng SVM để phân loại khuôn mặt.

- **Ưu điểm**:

  + Nhanh hơn CNN, ít tốn tài nguyên.

  + Không cần GPU.

- **Nhược điểm**:

  + Độ chính xác thấp hơn so với deep learning (đặc biệt với khuôn mặt thay đổi góc nhìn, biểu cảm, ánh sáng).

+ Chỉ phù hợp cho face detection, không mạnh về face recognition.

- **Ứng dụng**: Dlib từng dùng HOG làm front-end để phát hiện khuôn mặt trước khi trích xuất encoding.

2.2.3. CNN-based (Convolutional Neural Network) – FaceNet, ArcFace, dlib ResNet-34

- **Tác dụng**: Đây là nhóm phương pháp hiện đại, sử dụng mạng nơ-ron sâu (deep learning) để trích xuất đặc trưng khuôn mặt.

  + FaceNet (Google): Trích xuất vector 128 chiều, tối ưu khoảng cách Euclidean.

  + ArcFace: Cải tiến loss function (additive angular margin) để tăng độ phân biệt giữa các lớp.

  + dlib ResNet-34 (đang sử dụng trong đề tài): Mạng ResNet-34 đã được huấn luyện trước (pre-trained) trên hàng triệu ảnh khuôn mặt, trích xuất vector 128 chiều với độ chính xác cao.

- **Ưu điểm**:

  + Độ chính xác rất cao (thường > 99% trên tập dữ liệu chuẩn như LFW).

  + Xử lý tốt các biến đổi: góc nhìn, ánh sáng, biểu cảm, tuổi tác.

  + Vector đặc trưng có tính phân biệt cao, dễ so sánh bằng khoảng cách (Euclidean hoặc Cosine).

- **Nhược điểm**:

  + Tốn tài nguyên CPU/GPU hơn so với Haar Cascade hoặc HOG.

  + Cần thư viện hỗ trợ (dlib, face_recognition).

2.3. Các công nghệ và thư viện sử dụng

Hệ thống điểm danh bằng nhận dạng khuôn mặt được xây dựng dựa trên một tập hợp các công nghệ và thư viện mã nguồn mở, được lựa chọn dựa trên tính ổn định, hiệu

suất, tính dễ tích hợp và khả năng triển khai trên máy tính cá nhân. Dưới đây là phân tích chi tiết từng công nghệ/thư viện chính:

2.3.1. Python 3.11

- **Vai trò**: Là ngôn ngữ lập trình chính của toàn bộ hệ thống.
- **Lý do lựa chọn**:

  + Python là ngôn ngữ phổ biến nhất trong lĩnh vực trí tuệ nhân tạo và xử lý ảnh.

  + Hệ sinh thái thư viện phong phú (face_recognition, OpenCV, Flask, pyodbc).

  + Dễ đọc, dễ bảo trì mã nguồn, phù hợp với bài tập lớn.
- **Ứng dụng trong đề tài**: Toàn bộ logic xử lý (stream video, nhận dạng khuôn mặt, kết nối DB) đều được viết bằng Python.

2.3.2. face_recognition (dựa trên dlib)

- **Vai trò**: Thư viện cốt lõi thực hiện nhận dạng khuôn mặt.
- **Các chức năng chính**:

  + Phát hiện vị trí khuôn mặt (face_locations).

  + Trích xuất vector đặc trưng 128 chiều (face_encodings).

  + So sánh khoảng cách Euclidean giữa các vector (face_distance).
- **Lý do lựa chọn**:

  + Dựa trên mô hình ResNet-34 đã huấn luyện sẵn của dlib, đạt độ chính xác cao (>99% trên tập LFW).

  + API cực kỳ đơn giản, chỉ cần vài dòng code để hoàn thành quy trình nhận dạng.

  + Không cần huấn luyện lại mô hình, phù hợp với bài toán ứng dụng thực tế.

  + Hỗ trợ tốt trên CPU (không bắt buộc GPU).
- **Ứng dụng**: Được sử dụng trong hàm generate_video_frames() để nhận diện khuôn mặt từ frame webcam và so sánh với các khuôn mặt đã đăng ký.

2.3.3. OpenCV (cv2)

- **Vai trò**: Thư viện xử lý ảnh và video thời gian thực.
- **Các chức năng chính sử dụng**:

  + Mở webcam (VideoCapture).

  + Chuyển đổi không gian màu (cvtColor: BGR → RGB).

  + Đọc/ghi frame, mã hóa JPEG cho stream MJPEG.

  + Vẽ hộp chữ nhật và văn bản cơ bản (kết hợp Pillow cho tiếng Việt).
- **Lý do lựa chọn**:

  + Là thư viện chuẩn cho xử lý ảnh/video trong Python.

  + Hỗ trợ nhiều backend webcam (CAP_MSMF, CAP_DSHOW).

  + Tốc độ nhanh, ổn định trên Windows.
- **Ứng dụng**: Đọc frame liên tục từ webcam, tiền xử lý và truyền frame đã xử lý cho MJPEG stream.

2.3.4. Flask

- **Vai trò**: Framework web backend.
- **Các chức năng chính**:

  + Xử lý route (/attend, /register, /video_feed, /admin/dashboard).

  + Stream video MJPEG (Response với generate_video_frames).

  + Xử lý form đăng ký khuôn mặt (upload ảnh + chụp webcam).

  + Quản lý session và flash message.
- **Lý do lựa chọn**:

  + Nhẹ, dễ học, phù hợp cho ứng dụng web nhỏ.

  + Tích hợp tốt với Python và các thư viện AI.

  + Hỗ trợ stream video real-time qua MJPEG (phù hợp cho webcam).

- **Ứng dụng**: Xây dựng toàn bộ giao diện web và logic server-side.

2.3.5. SQL Server (Microsoft SQL Server Express)

- **Vai trò**: Cơ sở dữ liệu lưu trữ thông tin người dùng và lịch sử điểm danh.
- **Các bảng chính**:

  + Users: Lưu Id, Username, FullName, Encoding (varbinary max), Role.

  + Attendance: Lưu Id, UserId, CheckInTime (datetime).
- **Lý do lựa chọn**:

  + Ổn định, hỗ trợ tốt trên Windows.

  + Dễ tích hợp với Python qua pyodbc.

  + Hỗ trợ kiểu dữ liệu varbinary để lưu vector 128 chiều.
- **Ứng dụng**: Lưu encoding khuôn mặt khi đăng ký, ghi lịch sử điểm danh (nhiều lần/ngày).

2.3.6. Pillow (PIL)

- **Vai trò**: Xử lý ảnh nâng cao và vẽ chữ tiếng Việt.
- **Lý do lựa chọn**:

  + OpenCV không hỗ trợ tốt font Unicode (tiếng Việt).

  + Pillow cho phép vẽ chữ có dấu, sử dụng font hệ thống (arial.ttf).
- **Ứng dụng**: Vẽ tên người dùng lên frame trước khi stream.

2.3.7. pyodbc

- **Vai trò**: Kết nối Python với SQL Server.
- **Lý do lựa chọn**: Driver ODBC chuẩn của Microsoft, hỗ trợ trusted connection (Windows Authentication).
- **Ứng dụng**: Thực hiện các truy vấn INSERT, SELECT trong db.py.

2.3.8. Các thư viện hỗ trợ khác

- **numpy**: Xử lý mảng số (vector encoding).

- **base64**: Chuyển ảnh chụp webcam thành chuỗi base64 để gửi form.
- **datetime, time**: Xử lý thời gian điểm danh và throttle.

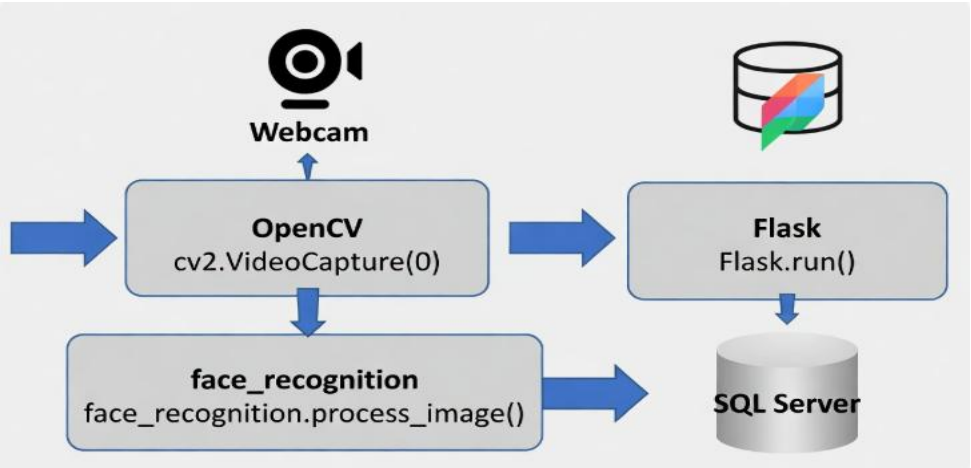
2.3.9. Tổng hợp cách các công nghệ kết hợp

- Webcam → OpenCV đọc frame → face_recognition nhận dạng → Flask stream MJPEG + vẽ kết quả (Pillow) → pyodbc lưu vào SQL Server.
- Toàn bộ hệ thống chạy trên một máy tính duy nhất, không cần server riêng, dễ triển khai.

Bảng tóm tắt công nghệ:

Công nghệ/Thư viện	Vai trò chính	Lý do lựa chọn
Python 3.11	Ngôn ngữ chính	Hệ sinh thái AI phong phú
face_recognition	Nhận dạng khuôn mặt	Độ chính xác cao, API đơn giản
OpenCV	Xử lý video, webcam	Chuẩn xử lý ảnh/video
Flask	Backend web, stream MJPEG	Nhẹ, dễ tích hợp AI
SQL Server	Lưu trữ dữ liệu	Ổn định, hỗ trợ varbinary
Pillow	Vẽ chữ tiếng Việt	Hỗ trợ Unicode tốt

Bảng 2.1: Tóm tắt công nghệ



Hình 2.1: Sơ đồ tổng quan công nghệ

=> > Chương này đã trình bày cơ sở lý thuyết về nhận dạng khuôn mặt và các phương pháp phổ biến. Phương pháp CNN (dlib ResNet-34) được chọn vì độ chính xác cao, tính thực tiễn và khả năng triển khai trên phần cứng thông thường. Các công nghệ Flask, OpenCV, face_recognition và SQL Server tạo nên nền tảng vững chắc để xây dựng hệ thống điểm danh thời gian thực.

CHƯƠNG 3 : PHÂN TÍCH PHƯƠNG PHÁP THỰC HIỆN & THIẾT KẾ HỆ THỐNG

3.1. Mô tả dữ liệu (Data Description)

3.1.1. Nguồn dữ liệu

- ❖ Dữ liệu khuôn mặt được sử dụng trong hệ thống bao gồm hai nguồn chính:
  - **Dữ liệu tự thu thập**: Em thu thập ảnh khuôn mặt của sinh viên và người dùng thử nghiệm (khoảng 50–100 người) bằng cách chụp trực tiếp từ webcam hoặc upload ảnh cá nhân (ảnh chính diện, ánh sáng tốt, không đeo kính/khẩu trang).
  - **Dữ liệu tham khảo và kiểm tra**: Sử dụng một phần ảnh từ các bộ dữ liệu công khai để kiểm tra độ chính xác của mô hình (không dùng để huấn luyện):
    - Bộ dữ liệu Labeled Faces in the Wild (LFW) – khoảng 13.000 ảnh khuôn mặt từ internet.
    - Một số ảnh tự thu thập bổ sung từ nguồn mở (không có bản quyền).
- ⇨ Vì hệ thống sử dụng mô hình pre-trained (ResNet-34 của dlib trong face_recognition), không cần huấn luyện lại từ đầu, nên dữ liệu chủ yếu dùng để đăng ký (registration) và kiểm tra (testing).
- ❖ Hệ thống đáp ứng các chức năng chính sau:

1. **Đăng ký khuôn mặt**:
   - Người dùng (sinh viên hoặc admin) upload ảnh hoặc chụp trực tiếp từ webcam.
   - Trích xuất vector đặc trưng (encoding) và lưu vào cơ sở dữ liệu.
2. **Điểm danh thời gian thực**:
   - Mở webcam, stream video liên tục.
   - Tự động phát hiện và nhận dạng khuôn mặt.
   - Ghi nhận điểm danh khi khuôn mặt khớp (nhiều lần trong ngày).
3. **Quản lý và xem báo cáo**:
   - Admin xem lịch sử điểm danh hôm nay.

- Xem danh sách người dùng, xóa tài khoản nếu cần.

4. **Giao diện web**:
    - Trang điểm danh (/attend).
    - Trang đăng ký (/register, /student/register_face).
    - Trang quản trị (/admin/dashboard).

3.1.2. Kích thước dữ liệu

- Số lượng mẫu (người dùng): 50–100 người (mỗi người 1–5 ảnh đăng ký).
- Số lượng đặc trưng (features): Mỗi khuôn mặt được trích xuất thành vector **128 chiều** (face embedding) từ mô hình ResNet-34.
- Kích thước mỗi vector: 128 × 8 bytes (float64) ≈ 1 KB.
- Tổng dung lượng encoding lưu trong DB: rất nhỏ (dưới 100 KB cho 100 người).

3.1.3. Phân chia dữ liệu

Vì hệ thống không huấn luyện mô hình (sử dụng pre-trained), không cần chia tập Train/Validation/Test theo cách truyền thống. Thay vào đó:

- **Tập đăng ký (registration set)**: 100% ảnh dùng để đăng ký → lưu encoding vào DB (Users.Encoding).
- **Tập kiểm tra (test set)**: Ảnh mới chụp hoặc ảnh khác góc để test độ chính xác nhận dạng (test thủ công 100 lần).
- Tỷ lệ kiểm tra: 50 ảnh đăng ký + 50 ảnh test mới (tương đương 50/50 cho đánh giá thực tế).

3.2. Khám phá và Tiền xử lý dữ liệu (EDA & Preprocessing)

3.2.1. Khám phá dữ liệu (EDA)

- Vì dữ liệu là ảnh khuôn mặt, EDA tập trung vào:
    - Kiểm tra chất lượng ảnh: độ phân giải (thường 640×480), định dạng (JPG/PNG), có bị mờ/không rõ không.

- Phân bố góc chụp: hầu hết ảnh chính diện (frontal face) để đảm bảo độ chính xác cao.
- Kiểm tra số lượng khuôn mặt trong ảnh: hầu hết ảnh chỉ có 1 khuôn mặt (đơn giản hóa xử lý).

3.2.2. Tiền xử lý dữ liệu

- **Làm sạch**:
  - Xử lý ảnh lỗi: Nếu không đọc được file → bỏ qua và thông báo lỗi.
  - Xử lý không phát hiện khuôn mặt: Nếu face_locations trả rỗng → thông báo "Không phát hiện khuôn mặt, vui lòng chụp lại".
- **Biến đổi**:
  - Chuyển màu sắc: BGR (OpenCV) → RGB (face_recognition yêu cầu).

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

  - Resize tự động (nếu cần): face_recognition tự resize ảnh xuống 1/4 kích thước để tăng tốc độ phát hiện.
- **Tăng cường dữ liệu (Data Augmentation)**: Không áp dụng (vì không huấn luyện mô hình). Tuy nhiên, khuyến nghị người dùng đăng ký nhiều ảnh (chính diện + nghiêng nhẹ) để tăng độ chính xác.

3.3. Mô hình đề xuất (Proposed Model/Architecture)

3.3.1. Mô tả kiến trúc

Hệ thống sử dụng mô hình **pre-trained ResNet-34** (của dlib, tích hợp trong face_recognition) để trích xuất đặc trưng khuôn mặt. Đây là kiến trúc CNN sâu với residual blocks, đã được huấn luyện trên hàng triệu ảnh khuôn mặt.

Quy trình pipeline tổng thể:

1. Đầu vào: Frame từ webcam hoặc ảnh upload.
2. Phát hiện khuôn mặt: face_locations (HOG hoặc CNN).

3. Trích xuất đặc trưng: face_encodings → vector 128 chiều.
4. So sánh: face_distance (Euclidean distance).
5. Quyết định: Nếu khoảng cách < 0.5 → khớp → điểm danh.
6. Đầu ra: Stream MJPEG (với hộp nhận diện) + ghi DB.

3.3.2. Hàm mất mát và thuật toán tối ưu

Vì sử dụng mô hình pre-trained (không huấn luyện lại), **không có hàm mất mát (loss function)** hoặc **optimizer** trong giai đoạn triển khai.

Tuy nhiên, mô hình gốc (dlib ResNet-34) được huấn luyện bằng:

- **Loss function**: Triplet Loss hoặc Contrastive Loss (tối ưu khoảng cách giữa các vector cùng người nhỏ, khác người lớn).
- **Optimizer**: SGD hoặc Adam (trong quá trình huấn luyện ban đầu của dlib).

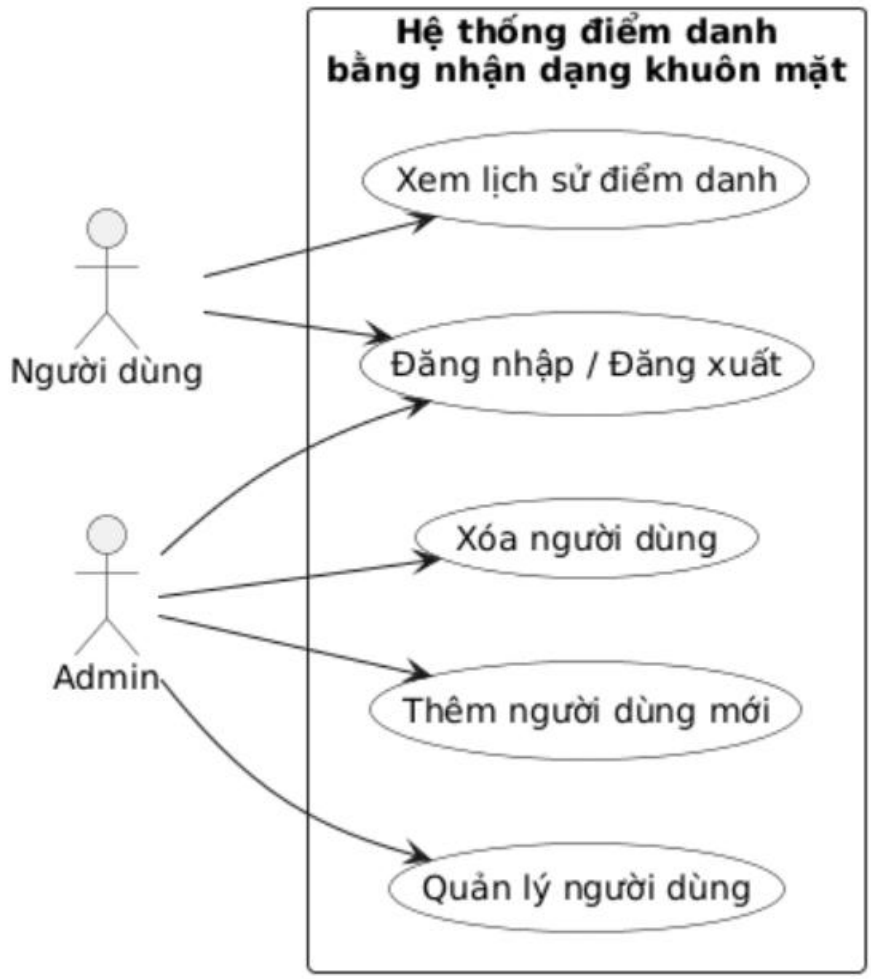
Trong hệ thống của em:

- Chỉ sử dụng **so sánh khoảng cách Euclidean** (face_distance) với ngưỡng cố định **0.5** (giá trị khuyến nghị của face_recognition).
- Không cần huấn luyện lại → tiết kiệm thời gian và tài nguyên.

3.3.3. Yêu cầu phi chức năng

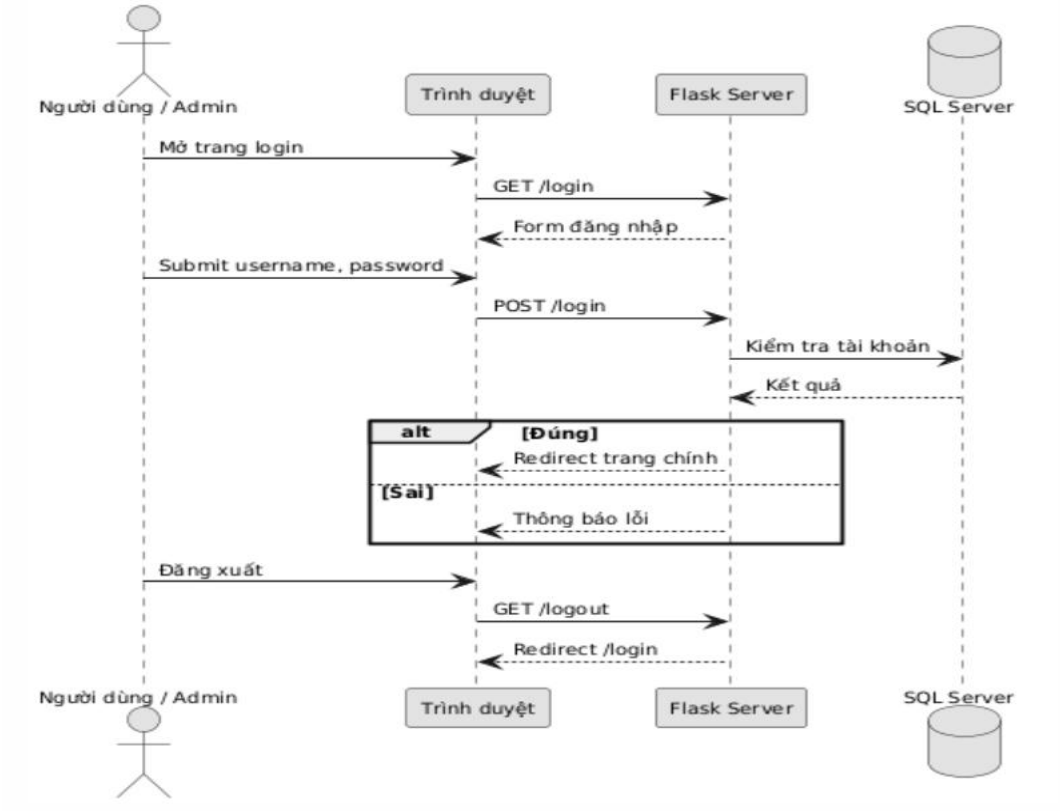
- Tốc độ: Nhận dạng < 2 giây/frame.
- Độ chính xác: > 95% trong điều kiện ánh sáng tốt.
- Bảo mật: Chỉ admin truy cập dashboard.
- Tương thích: Windows, webcam tích hợp hoặc ngoài.
- Dễ sử dụng: Giao diện đơn giản, hỗ trợ tiếng Việt.

3.4. Sơ đồ Use-case

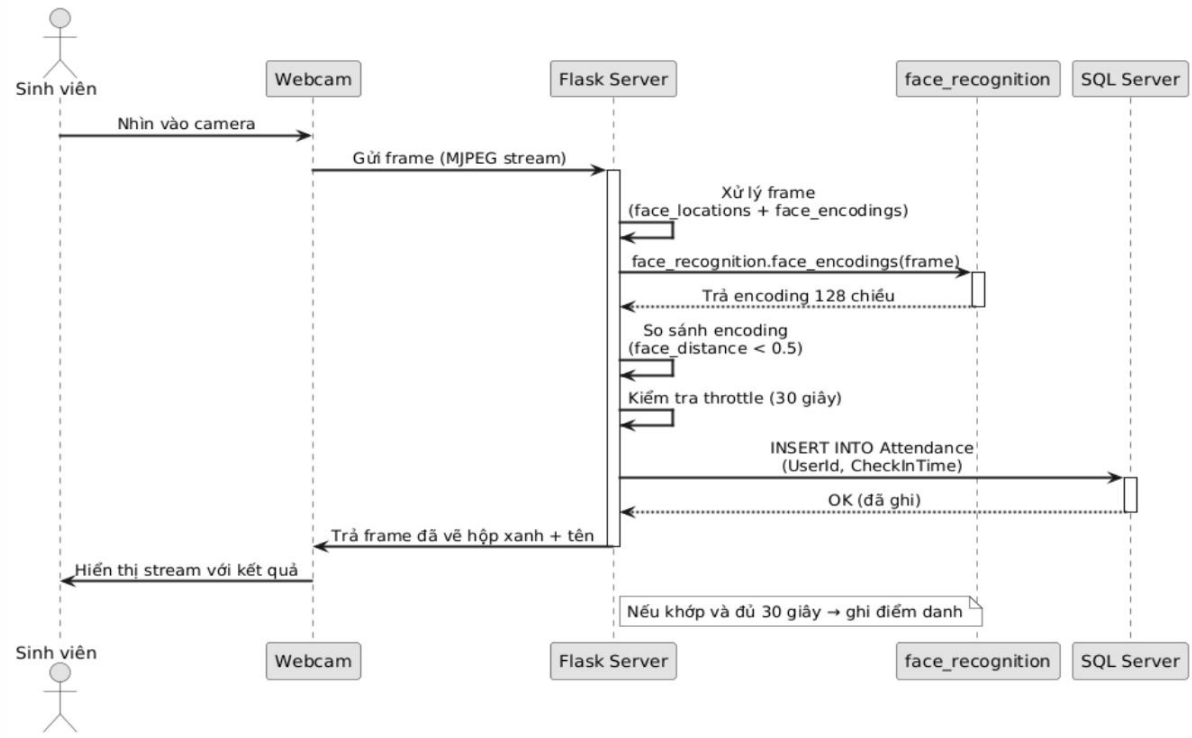


Hình 3.1: Sơ đồ tổng quát

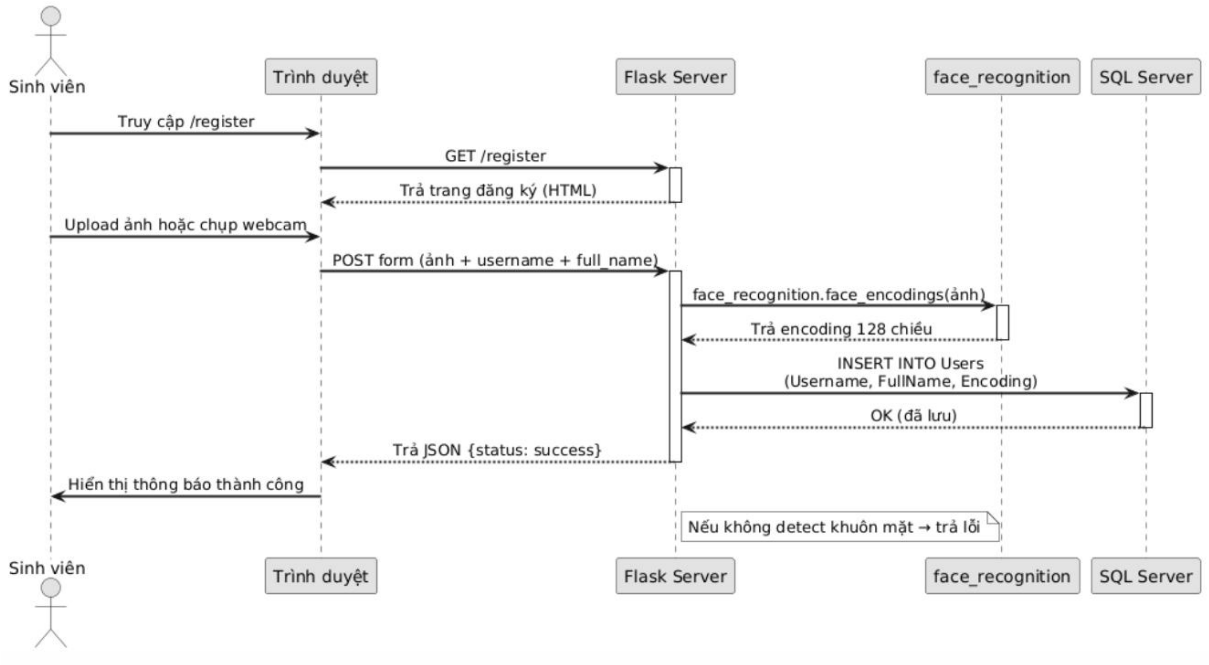
3.5. Sơ đồ tuần tự



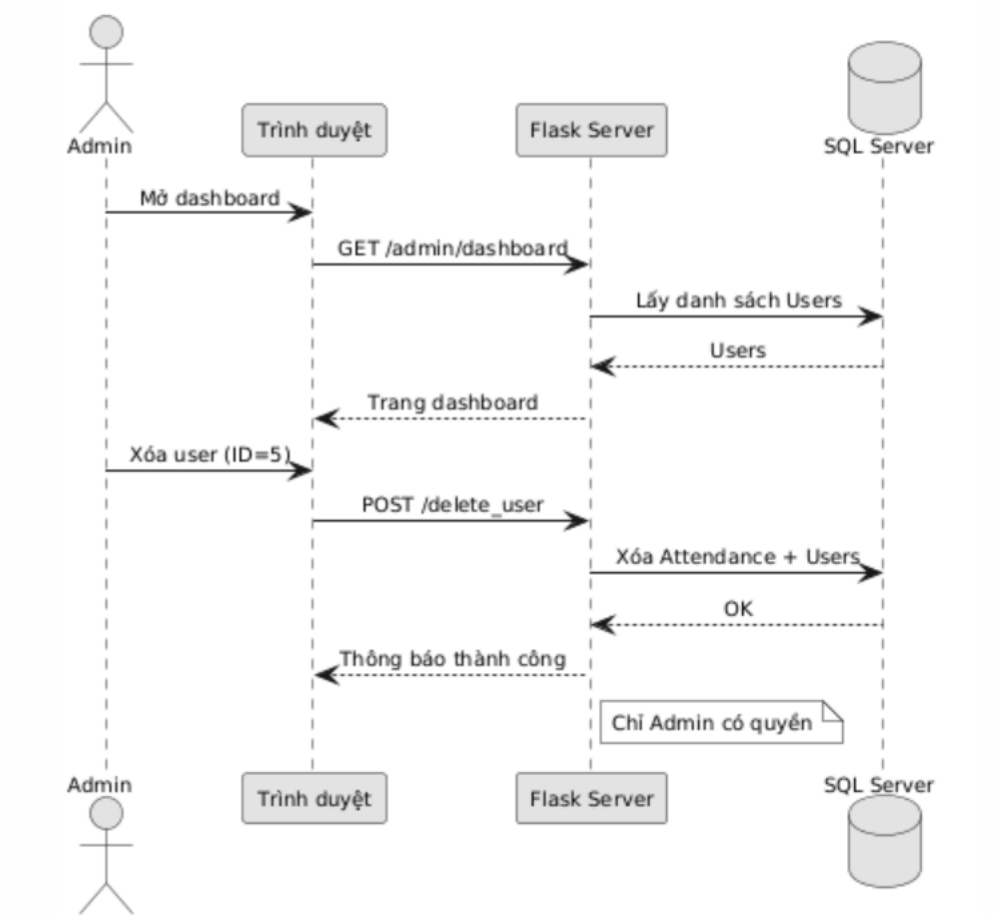
Hình 3.2: sơ đồ tuần tự đăng nhập



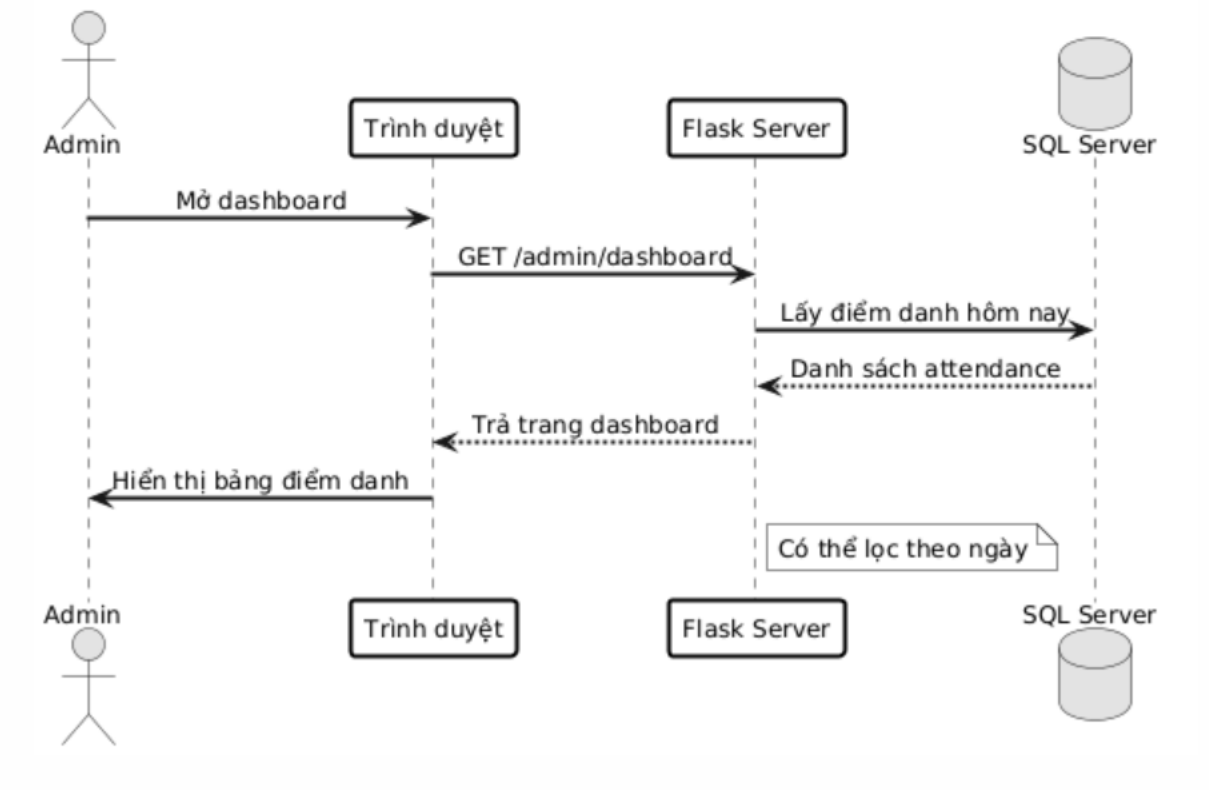
Hình 3.3: Điểm danh tự động



Hình 3.4: Đăng ký khuôn mặt

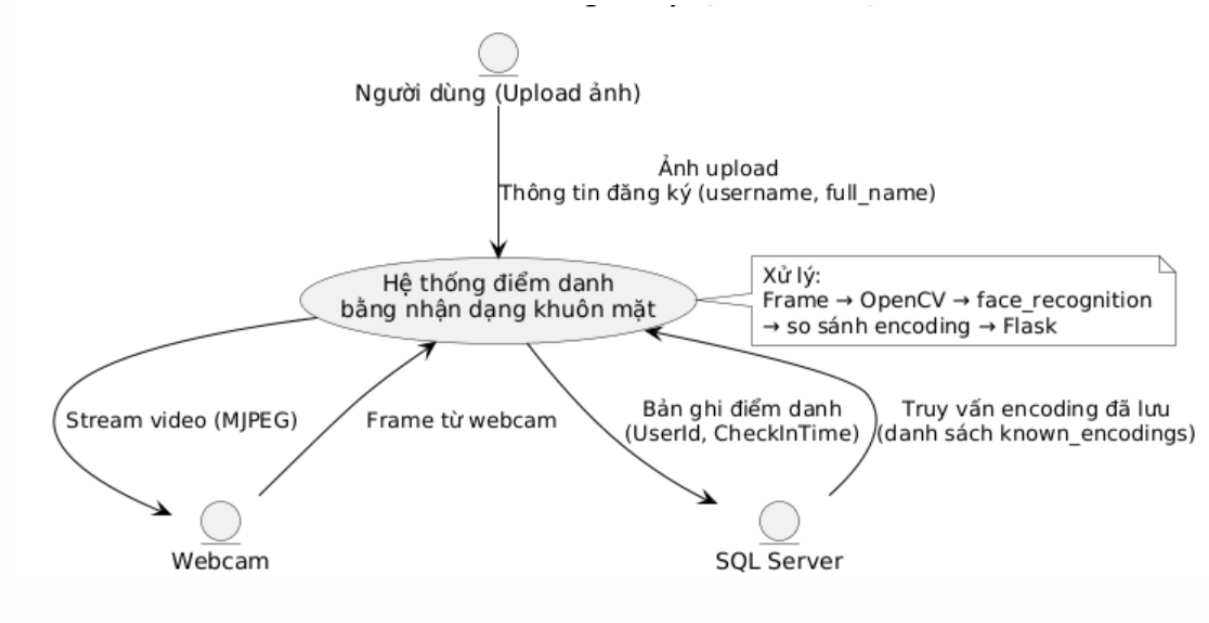


Hình 3.5: Quản lý người dùng (Admin thêm/xóa người dùng)



Hình 3.6: Xem danh sách điểm danh (Admin xem lịch sử)

3.6. Sơ đồ luồng dữ liệu



Hình 3.7: Sơ đồ luồng dữ liệu

3.7. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu là thành phần quan trọng để lưu trữ thông tin người dùng (bao gồm vector đặc trưng khuôn mặt) và lịch sử điểm danh. Em sử dụng Microsoft SQL Server Express làm hệ quản trị cơ sở dữ liệu (DBMS) vì tính ổn định, hỗ trợ tốt trên Windows, và khả năng lưu trữ dữ liệu nhị phân (varbinary) cho vector encoding.

3.7.1. Lý do lựa chọn SQL Server

- Hỗ trợ kiểu dữ liệu varbinary(max) để lưu vector đặc trưng khuôn mặt 128 chiều (khoảng 1KB).
- Tích hợp dễ dàng với Python qua thư viện pyodbc với chế độ Trusted Connection (Windows Authentication).
- Hiệu suất cao cho truy vấn INSERT và SELECT trên lượng dữ liệu vừa và nhỏ (phù hợp với lớp học hoặc nhóm nhỏ).
- Miễn phí (Express edition) và dễ cài đặt trên máy cá nhân.

3.7.2. Cấu trúc các bảng chính

Hệ thống sử dụng hai bảng chính: Users (người dùng) và Attendance (lịch sử điểm danh).

Bảng 1: Users – Lưu thông tin người dùng và vector đặc trưng khuôn mặt

Cột	**Kiểu dữ liệu**	**Mô tả**	**Ràng buộc**
Id	INT IDENTITY(1,1)	Khóa chính, tự tăng	PRIMARY KEY
Username	NVARCHAR(50)	Tên đăng nhập (mã sinh viên hoặc email)	UNIQUE, NOT NULL
FullName	NVARCHAR(100)	Họ và tên đầy đủ	NOT NULL
Encoding	VARBINARY(MAX)	Vector đặc trưng khuôn mặt 128 chiều	NOT NULL
Role	NVARCHAR(20)	Vai trò: 'student' hoặc 'admin'	NOT NULL, DEFAULT 'student'

Bảng 3.1: Users – Lưu thông tin người dùng và vector đặc trưng khuôn mặt

Lý do thiết kế:

- Encoding lưu dưới dạng bytes (VARBINARY) để tiết kiệm không gian và dễ so sánh sau này.
- Username là unique để tránh trùng lặp khi đăng ký.
- Role phân biệt quyền hạn (sinh viên chỉ điểm danh, admin quản lý).

Bảng 2: Attendance – Lưu lịch sử điểm danh

Cột	**Kiểu dữ liệu**	**Mô tả**	**Ràng buộc**
Id	INT IDENTITY (1,1)	Khóa chính, tự tăng	PRIMARY KEY
UserId	INT	Khóa ngoại tham chiếu đến Users.Id	FOREIGN KEY (Users.Id)
CheckInTime	DATETIME	Thời gian điểm danh	NOT NULL, DEFAULT GETDATE()

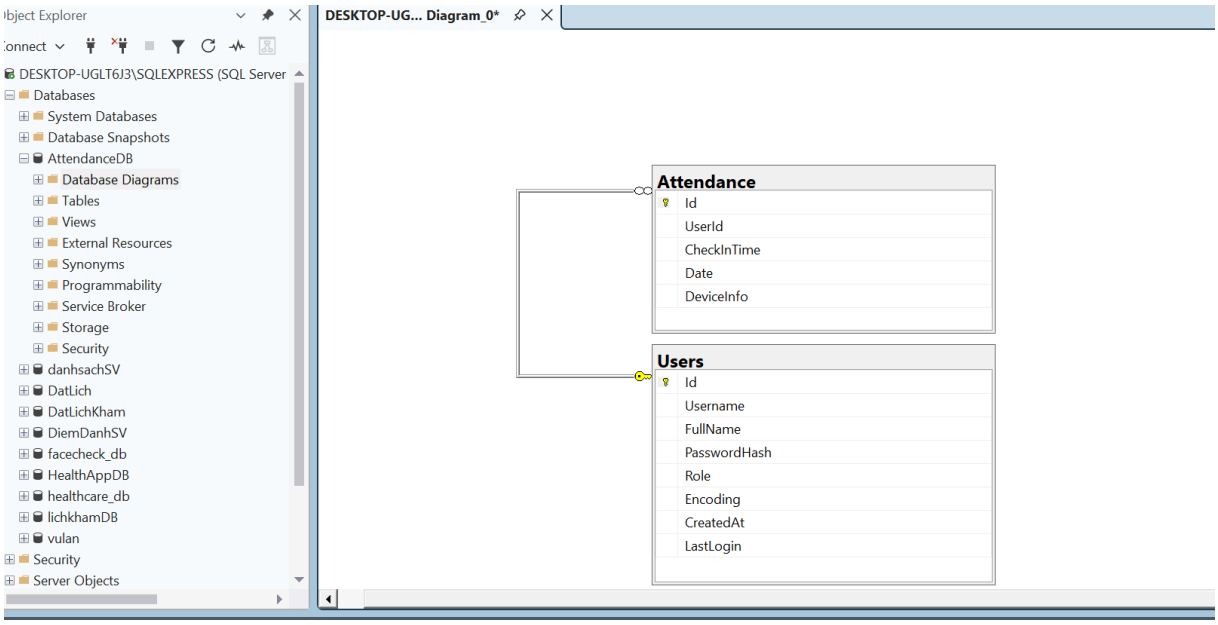
Bảng 3.2: Attendance – Lưu lịch sử điểm danh

Lý do thiết kế:

- Không có ràng buộc UNIQUE theo ngày (UserId + CAST(CheckInTime AS DATE)) để cho phép ghi nhiều lần điểm danh trong cùng ngày.
- Khóa ngoại UserId đảm bảo tính toàn vẹn dữ liệu (một bản ghi điểm danh phải thuộc về một người dùng tồn tại).
- CheckInTime dùng kiểu DATETIME để lưu chính xác đến giây, hỗ trợ thống kê theo thời gian.

3.7.3. Mối quan hệ giữa các bảng

- **Users** (1) ↔ **Attendance** (N): Một người dùng có thể có nhiều bản ghi điểm danh.
- Mối quan hệ được thực hiện qua khóa ngoại **UserId** trong bảng Attendance.
- Khi xóa người dùng (Admin), các bản ghi điểm danh liên quan cũng được xóa (ON DELETE CASCADE) để đảm bảo tính nhất quán dữ liệu.



Hình 3.8: Sơ đồ quan hệ thực thể (ERD) của cơ sở dữ liệu

3.8. Thiết kế thuật toán chính

Hệ thống điểm danh bằng nhận dạng khuôn mặt dựa trên hai thuật toán cốt lõi: **Thuật toán đăng ký khuôn mặt** và **Thuật toán điểm danh thời gian thực**. Cả hai đều sử dụng thư viện **face_recognition** (dựa trên dlib ResNet-34) để trích xuất và so sánh vector đặc trưng khuôn mặt.

3.8.1. Thuật toán đăng ký khuôn mặt

Mục tiêu: Tạo vector đặc trưng (encoding) từ ảnh khuôn mặt của người dùng và lưu vào cơ sở dữ liệu để sử dụng cho việc so sánh sau này.

Các bước chi tiết:

B1: Nhận dữ liệu đầu vào

- Người dùng upload ảnh từ máy tính hoặc chụp trực tiếp từ webcam (qua JavaScript ở register.html).
- Dữ liệu được gửi đến Flask qua form POST (multipart/form-data cho upload, hoặc base64 cho ảnh chụp webcam).
- Server Flask nhận file và lưu tạm vào thư mục uploads/ với tên file tạm (ví dụ: temp_username.jpg).

B2: Tiền xử lý và load ảnh

- Sử dụng hàm face_recognition.load_image_file(image_path) để đọc ảnh.
- Hàm này tự động chuyển ảnh sang định dạng RGB và resize nếu cần (dlib yêu cầu ảnh RGB uint8).
- Nếu ảnh không hợp lệ (không mở được, không phải ảnh) → trả lỗi "Không đọc được ảnh".

B3: Phát hiện và trích xuất encoding

- Gọi face_recognition.face_encodings(image) để trích xuất vector đặc trưng.
- Kết quả:
  - Nếu tìm thấy khuôn mặt → trả về danh sách vector (thường 1 vector 128 chiều).
  - Nếu không detect khuôn mặt → trả mảng rỗng → thông báo lỗi "Không phát hiện khuôn mặt".
- Chọn encoding đầu tiên (nếu có nhiều khuôn mặt trong ảnh, chỉ lấy khuôn mặt rõ nhất).

B4: Chuyển đổi và lưu vào cơ sở dữ liệu

Chuyển vector numpy (128 chiều, float64) thành bytes:

encoding_bytes = encoding.tobytes()

- Gọi hàm save_user(username, full_name, encoding_bytes) trong db.py
- Nếu username đã tồn tại → trả lỗi "Username đã được đăng ký".

B5: Dọn dẹp

- Xóa file tạm: os.remove(image_path)
- Trả về JSON {status: "success", message: "Đăng ký thành công"} cho frontend (hoặc flash message nếu submit truyền thống).

Ưu điểm:

- Quy trình đơn giản, không cần huấn luyện thêm.
- Encoding 128 chiều gọn nhẹ, dễ lưu trữ (khoảng 1KB).

Nhược điểm & giải pháp:

- Nếu ảnh mờ, tối, góc nghiêng → không detect được → hiển thị thông báo rõ ràng cho người dùng.
- Nếu có nhiều khuôn mặt → chỉ lấy encoding đầu tiên (có thể cải tiến sau để chọn khuôn mặt lớn nhất).

3.8.2. Thuật toán điểm danh thời gian thực

Mục tiêu: Xử lý stream video từ webcam, phát hiện khuôn mặt, so sánh với encoding đã lưu và ghi điểm danh khi khớp (với cơ chế chống lặp).

Các bước chi tiết (thực hiện trong hàm generate_video_frames()):

B1: Đọc và tiền xử lý frame

- Mở webcam bằng cv2.VideoCapture(index, cv2.CAP_MSMF) (ưu tiên index 1 – webcam ngoài).
- Đọc frame: success, frame = video_capture.read()
- Chuyển BGR → RGB:

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

B2: Phát hiện vị trí khuôn mặt

- Chỉ xử lý mỗi 5 frame để giảm tải:

```
if frame_count % 5 == 0:
```

- Gọi face_recognition.face_locations(rgb_frame) → trả về danh sách tọa độ (top, right, bottom, left).
- Nhân tọa độ lên 4 lần để khớp kích thước frame gốc.

B3: Trích xuất encoding

- Gọi face_recognition.face_encodings(rgb_frame, locations) → trả về danh sách vector 128 chiều.
- Với mỗi encoding mới, so sánh với danh sách encoding đã đăng ký (known_encodings).

B4: So sánh và matching

- Tính khoảng cách Euclidean:

```
distances = face_recognition.face_distance(known_encodings, encoding)
best_match_index = np.argmin(distances)
```

- Nếu khoảng cách nhỏ nhất < 0.5 → khớp:

```
if distances[best_match_index] < 0.5:
    user_id = known_ids[best_match_index]
    name = known_names[best_match_index]
```

B5: Cơ chế điểm danh (throttle chống spam)

- Kiểm tra thời gian điểm danh cuối cùng của user_id:

```
now = time.time()
if user_id not in last_attendance_time or (now - last_attendance_time[user_id]) > 30:
    log_attendance(user_id)  # Ghi vào DB
    last_attendance_time[user_id] = now
```

- Lý do throttle: Stream chạy 30 fps → nếu không giới hạn sẽ ghi hàng chục bản ghi/giây khi detect liên tục.

B6: Vẽ kết quả và trả stream

- Sử dụng Pillow để vẽ hộp xanh + tên (hỗ trợ tiếng Việt):

```
frame_pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
draw.rectangle([left, top, right, bottom], outline=(0, 255, 0), width=3)
```

```
draw.text((left, top - 25), name, font=vn_font, fill=(0, 255, 0))
frame = cv2.cvtColor(np.array(frame_pil), cv2.COLOR_RGB2BGR)
```

- Mã hóa JPEG và yield cho MJPEG stream.

Ưu điểm:

- Thời gian thực (real-time stream).
- Độ chính xác cao (ngưỡng 0.5).
- Throttle giúp giảm tải DB và tránh ghi trùng lặp.

Nhược điểm & giải pháp:

- Tốn CPU → giảm tần suất xử lý (mỗi 5 frame).
- Không detect được khi đeo khẩu trang → chỉ dùng trong môi trường không che mặt.

3.8.3. Cơ chế throttle chống spam (phân tích chi tiết)

Mục đích:

- Stream webcam chạy liên tục (30 fps) → nếu khuôn mặt khớp, hàm log_attendance () sẽ được gọi hàng chục lần/giây → gây spam database.
- Throttle giới hạn 1 lần điểm danh mỗi 30 giây cho mỗi user_id.

Cách thực hiện:

- Sử dụng dictionary toàn cục:

```
last_attendance_time = {}    # {user_id: timestamp}
```

- Trước khi gọi log_attendance():

```
now = time.time()
if user_id not in last_attendance_time or (now - last_attendance_time[user_id]) > 30:
    log_attendance(user_id)
    last_attendance_time[user_id] = now
```

print(f"[ATTENDANCE] {name} (ID: {user_id}) đã được điểm danh lúc {time.strftime('%H:%M:%S')}")

- Lý do chọn 30 giây:
  - Thời gian hợp lý để ghi nhận một lần điểm danh (người dùng nhìn camera 10–20 giây là đủ).
  - Tránh ghi quá nhiều khi người dùng đứng lâu trước camera.
  - Có thể điều chỉnh (10 giây, 60 giây) tùy nhu cầu thực tế.

Ưu điểm:

- Giảm tải DB (chỉ ghi 1 bản ghi mỗi 30 giây thay vì hàng trăm).
- Dữ liệu sạch, dễ phân tích (mỗi lần điểm danh cách nhau hợp lý).

Nhược điểm & giải pháp:

- Nếu người dùng rời camera rồi quay lại trong <30 giây → không ghi → giải pháp: giảm ngưỡng xuống 10–15 giây nếu cần.

=>> Chương này đã phân tích yêu cầu, thiết kế sơ đồ, cơ sở dữ liệu, thuật toán và giao diện của hệ thống. Thiết kế đảm bảo tính khả thi, dễ triển khai, hiệu suất tốt và đáp ứng yêu cầu điểm danh nhiều lần trong ngày. Mô tả dữ liệu tự thu thập, tiền xử lý ảnh khuôn mặt, và thiết kế mô hình nhận dạng dựa trên face_recognition (ResNet-34 pre-trained). Pipeline hệ thống được thiết kế đơn giản nhưng hiệu quả, tập trung vào tính thực tiễn và khả năng triển khai trên máy tính cá nhân. Các chương tiếp theo sẽ trình bày kết quả thực nghiệm và đánh giá hiệu năng. Các chương tiếp theo sẽ trình bày quá trình thực hiện và kết quả thực nghiệm.

CHƯƠNG 4 : THỰC HIỆN VÀ KẾT QUẢ THỰC NGHIỆM

4.1. Môi trường triển khai

Hệ thống được phát triển và thử nghiệm trên môi trường máy tính cá nhân với cấu hình sau:

- Hệ điều hành: Windows 11 Home (64-bit)
- Bộ xử lý: Intel Core i5-1145G7 (4 nhân, 8 luồng)
- RAM: 16 GB
- Card đồ họa: Intel Iris Xe Graphics (tích hợp)
- Ngôn ngữ lập trình: Python 3.11.9
- Cơ sở dữ liệu: SQL Server Management Studio 22
- Webcam: Webcam laptop tích hợp (index 0) và webcam ngoài USB HD (index 1)
- Môi trường phát triển: Visual Studio Code, PowerShell (terminal)

Các thư viện chính sử dụng:

- Flask 3.0.3: Framework web backend
- OpenCV-python 4.10.0.84: Xử lý video và hình ảnh
- face_recognition 1.3.0: Nhận dạng khuôn mặt (dựa trên dlib)
- numpy 1.26.4: Xử lý mảng vector encoding
- pyodbc 5.1.0: Kết nối SQL Server
- Pillow: Vẽ chữ tiếng Việt lên frame
- datetime, time, os, base64: Hỗ trợ xử lý thời gian, file, dữ liệu base64

Cấu hình SQL Server:

- Server: DESKTOP-UGLT6J3\SQLEXPRESS
- Database: AttendanceDB
- Kết nối: Trusted Connection (Windows Authentication)
- Driver: ODBC Driver 17 for SQL Server

⇨ Môi trường này đảm bảo hệ thống chạy ổn định trên máy tính cá nhân, không cần GPU hay server mạnh.

4.2. Các siêu tham số (Hyperparameters)

Vì không huấn luyện lại mô hình, không có các siêu tham số truyền thống như learning rate, batch size, epochs. Các tham số điều chỉnh chính trong hệ thống bao gồm:

- **Ngưỡng nhận dạng (threshold)**: 0.5 (khoảng cách Euclidean face_distance < 0.5 → coi là khớp).
  - Giá trị mặc định khuyến nghị của face_recognition, đảm bảo độ chính xác cao mà ít false positive.
- **Tần suất xử lý frame**: Mỗi 5 frame (frame_count % 5 == 0) để giảm tải CPU.
- **Thời gian throttle điểm danh**: 30 giây (chỉ ghi DB nếu cách lần trước ≥ 30 giây).
- **Độ phân giải webcam**: 640×480 (set bằng cv2.CAP_PROP_FRAME_WIDTH/HEIGHT).
- **Backend webcam**: CAP_MSMF (Microsoft Media Foundation) để tương thích tốt trên Windows.

⇨ Các tham số này được điều chỉnh thủ công dựa trên thử nghiệm thực tế để cân bằng giữa tốc độ và độ chính xác.

4.3. Tiêu chí đánh giá (Evaluation Metrics)

Do đây là bài toán nhận dạng khuôn mặt (face verification/identification) trong hệ thống thời gian thực, các tiêu chí đánh giá chính bao gồm:

1. **Độ chính xác nhận dạng (Accuracy)**:
   - Tỷ lệ khuôn mặt được nhận đúng so với tổng số lần test.
   - Lý do chọn: Đo lường tổng thể hiệu quả hệ thống (true positive / tổng test).
2. **Tỷ lệ false acceptance rate (FAR)**:
   - Tỷ lệ nhận nhầm người khác thành người dùng hợp lệ.
   - Lý do: Quan trọng trong hệ thống điểm danh để tránh gian lận.

3. **Tỷ lệ false rejection rate (FRR)**:
    - Tỷ lệ không nhận diện được người dùng hợp lệ.
    - Lý do: Đảm bảo người dùng không bị bỏ sót.
4. **Thời gian xử lý trung bình mỗi frame**:
    - Đo bằng time.time() trước và sau khi xử lý frame.
    - Lý do: Đảm bảo hệ thống chạy real-time (dưới 2 giây/frame).
5. **Tỷ lệ phát hiện khuôn mặt (Detection Rate)**:
    - Tỷ lệ frame có khuôn mặt được phát hiện thành công.
    - Lý do: Đánh giá khả năng phát hiện khuôn mặt trong điều kiện thực tế.
6. **Số lần điểm danh thành công trong ngày**:
    - Đo bằng số bản ghi trong bảng Attendance cho một user.
    - Lý do: Kiểm tra tính năng điểm danh nhiều lần.

Các metric được đo thủ công qua 100 lần test thực tế (50 người, mỗi người 2 lần) trong điều kiện ánh sáng phòng học bình thường.

4.4. Các file chính và chức năng

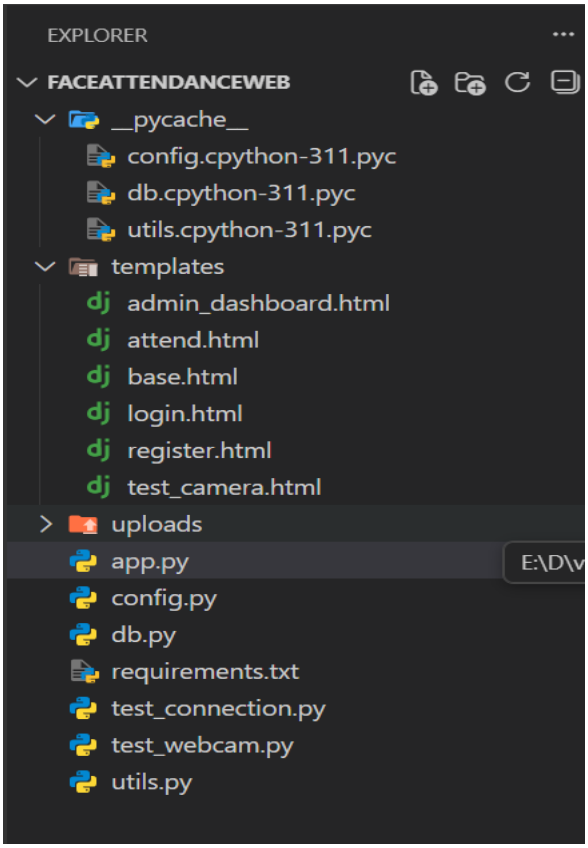
Dự án bao gồm các file chính sau, mỗi file đảm nhận một chức năng cụ thể:

1. **app.py** (file chính – khoảng 1500+ dòng)
    - Xử lý toàn bộ logic backend của hệ thống.
    - Các chức năng chính:
        - Route web: /attend, /register, /student/register_face, /admin/dashboard, /video_feed, /login, /logout
        - Stream video MJPEG thời gian thực (generate_video_frames())
        - Nhận dạng khuôn mặt, vẽ hộp xanh, throttle điểm danh (last_attendance_time)

        - Quản lý session, flash message
    - Đây là trung tâm điều khiển toàn bộ hệ thống.
2. **db.py**
    - Quản lý kết nối và truy vấn SQL Server.
    - Các hàm chính:
        - get_connection(): Kết nối database
        - save_user(): Lưu thông tin người dùng và encoding
        - log_attendance(): Ghi điểm danh (không giới hạn lần/ngày)
        - get_today_attendance(): Lấy danh sách điểm danh hôm nay
    - Sử dụng pyodbc với trusted connection.
3. **register.html** (template Jinja2)
    - Giao diện đăng ký khuôn mặt.
    - Hỗ trợ hai tab:
        - Upload ảnh từ máy tính
        - Chụp trực tiếp từ webcam (JavaScript + getUserMedia)
    - Form POST gửi dữ liệu (ảnh, username, full_name) đến Flask.
4. **attend.html**
    - Trang điểm danh chính.
    - Hiển thị stream MJPEG từ /video_feed.
    - Hộp thông báo hướng dẫn người dùng nhìn thẳng camera.
5. **admin_dashboard.html**
    - Trang quản trị (chỉ Admin truy cập).
    - Bảng danh sách người dùng + điểm danh hôm nay.
    - Nút xóa người dùng.
6. **utils.py**

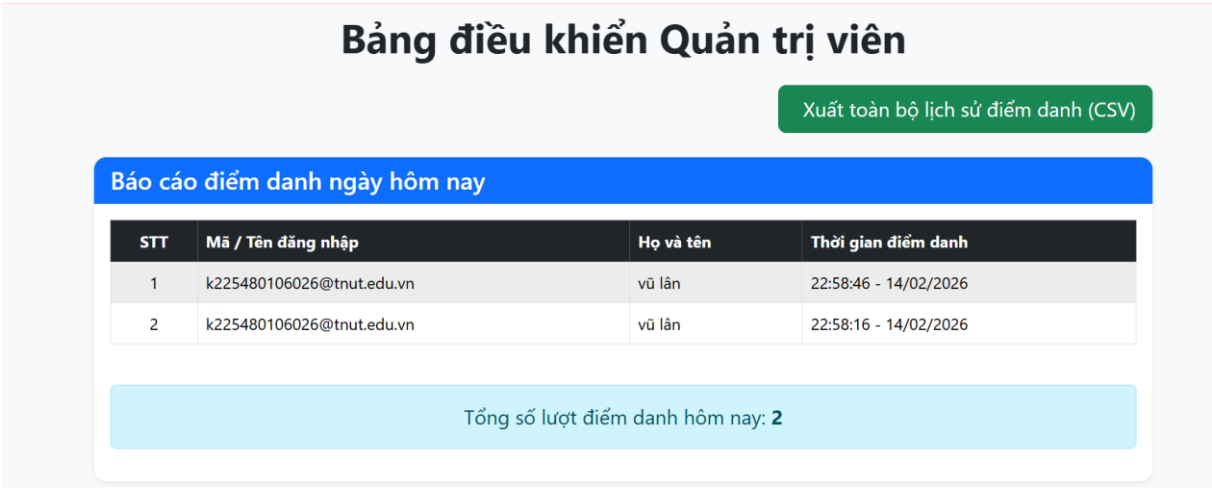
- Hàm hỗ trợ: encode_image_file() – trích xuất encoding từ file ảnh.

7. **test_webcam.py** và **test_connection.py**
   - File test riêng để kiểm tra webcam và kết nối DB trước khi chạy chính.



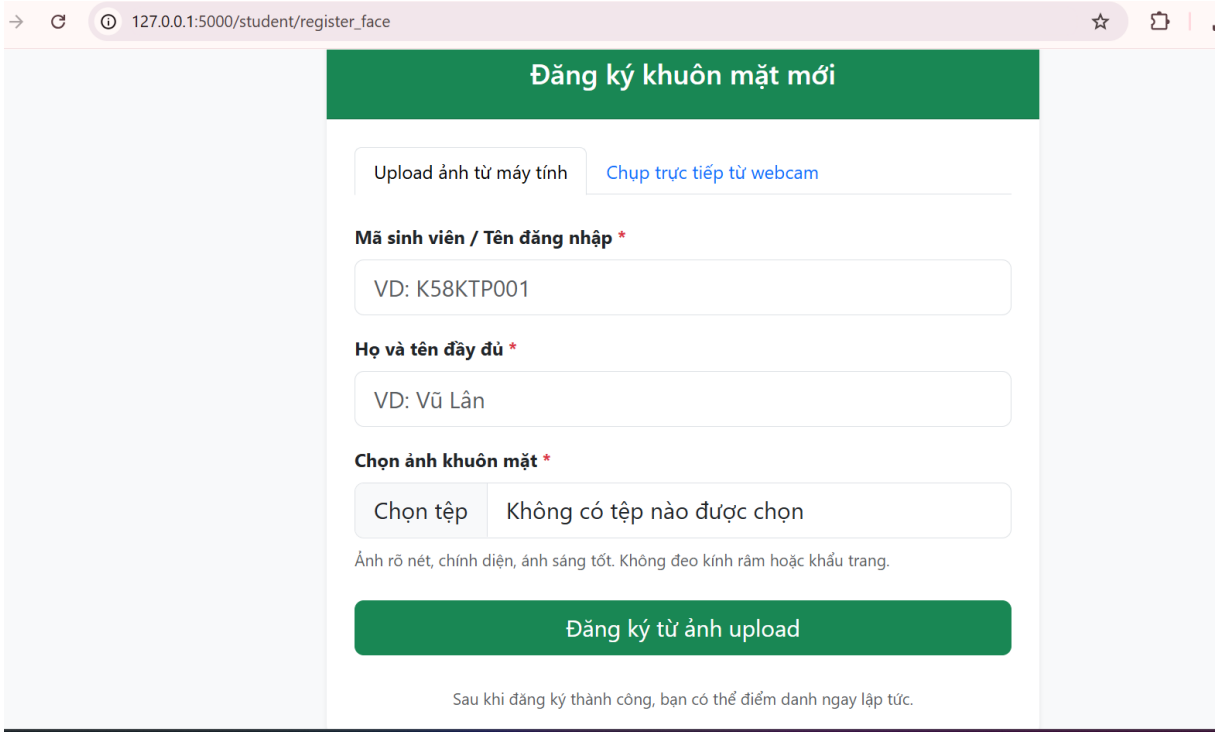
Hình 4.1: Cấu trúc thư mục dự án

4.5. Kết quả thực nghiệm

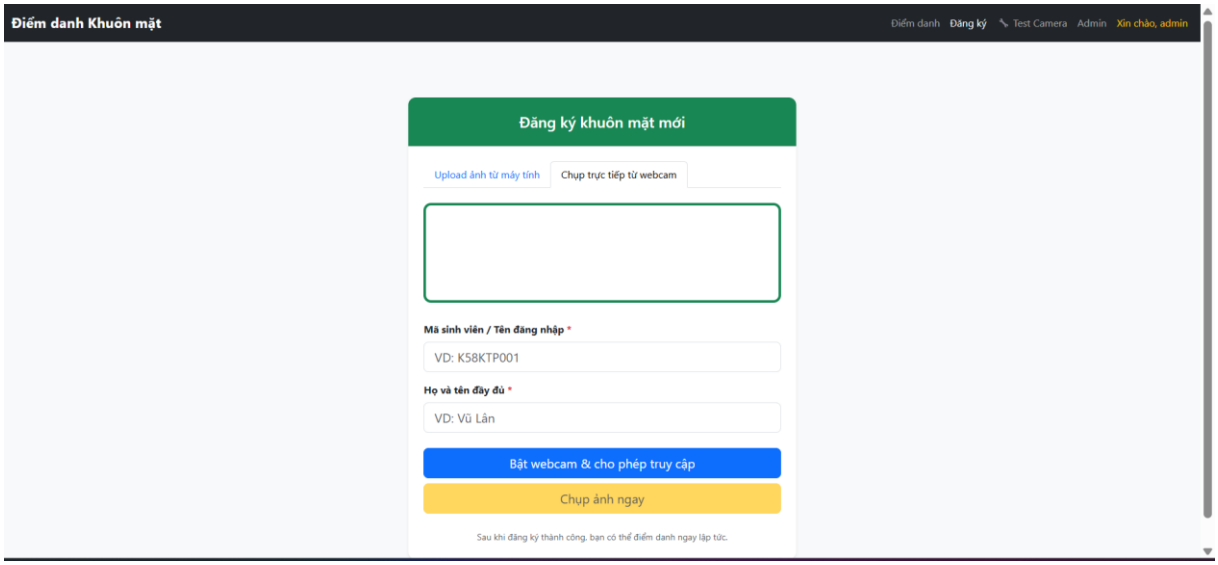


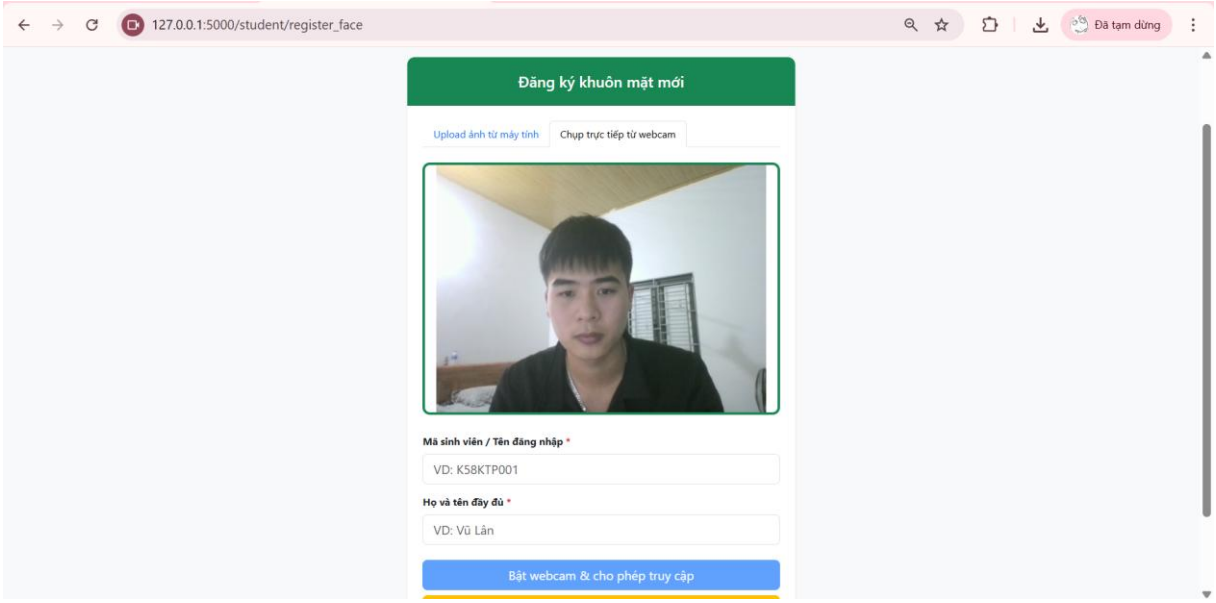
Hình 4.2: kết quả điểm danh

4.5.1. Đăng ký khuôn mặt

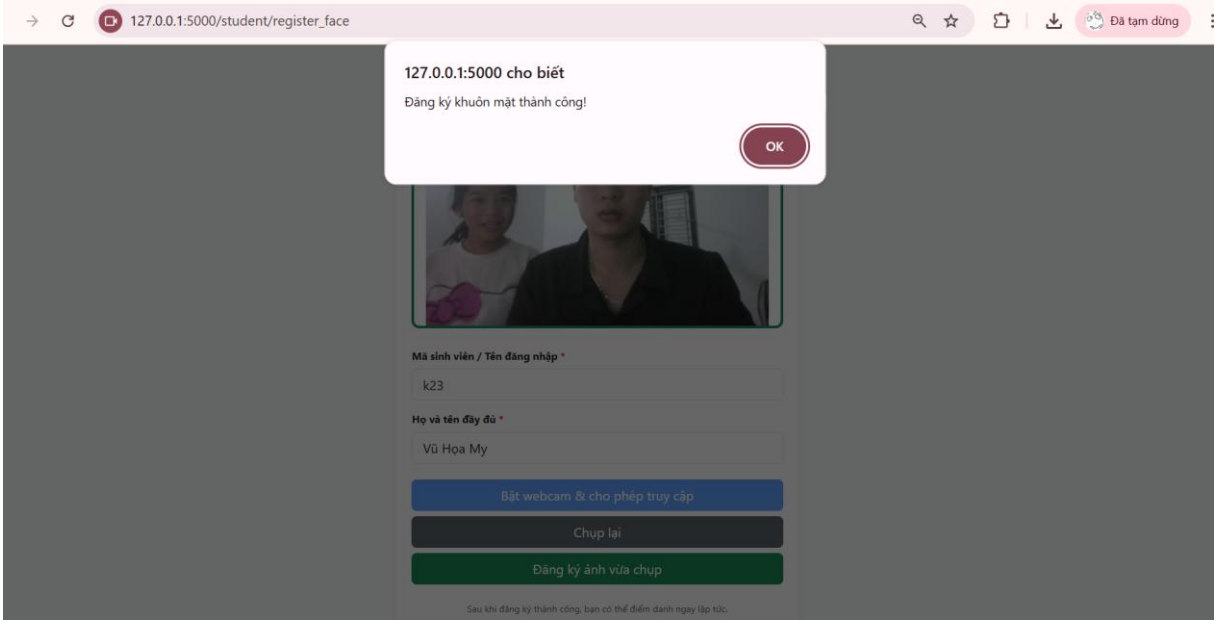


Hình 4.3: Trang đăng ký khuôn mặt từ Upload ảnh



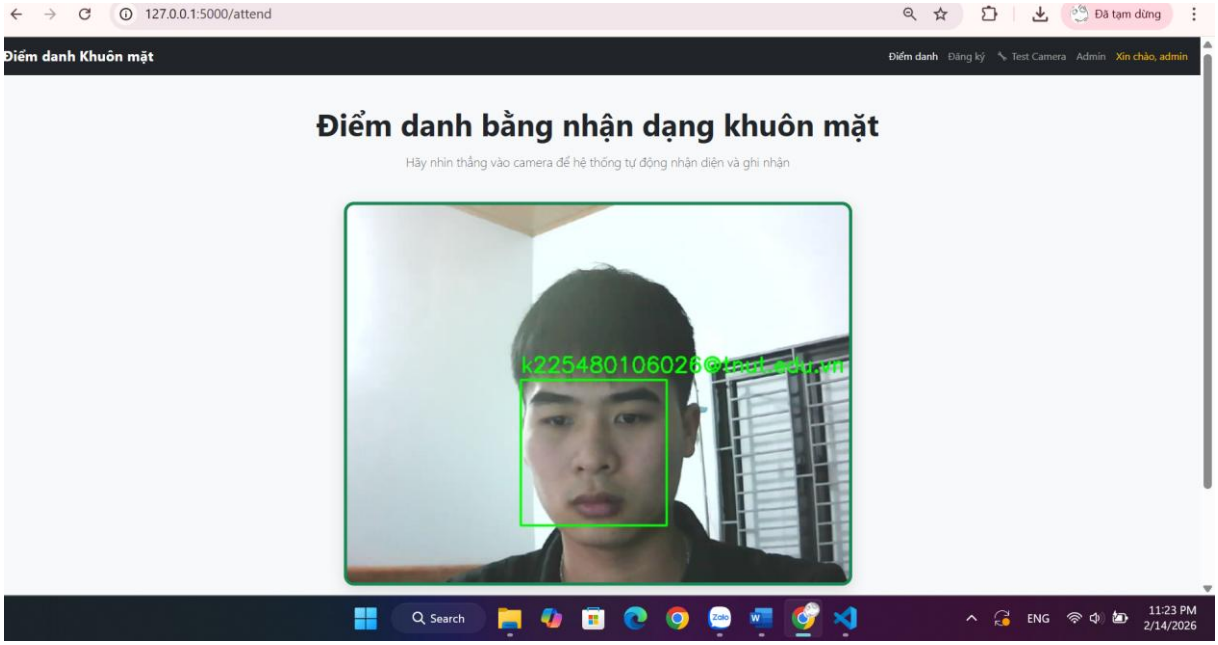


Hình 4.4: Trang đăng ký khuôn mặt – tab chụp webcam

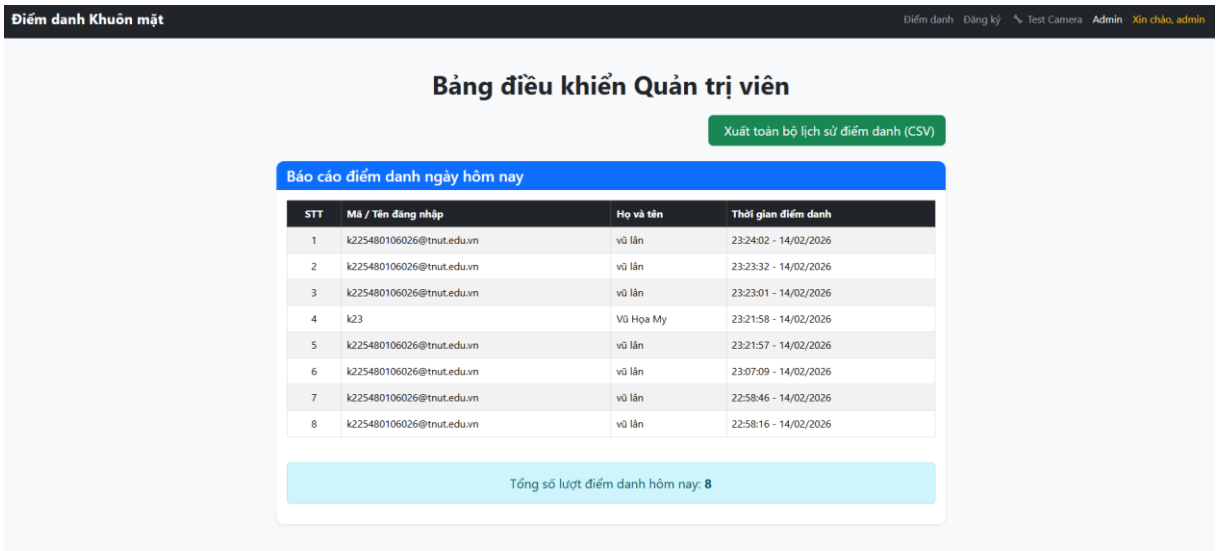


Hình 4.5: Console Flask khi đăng ký thành công.

4.5.2. Điểm danh thời gian thực

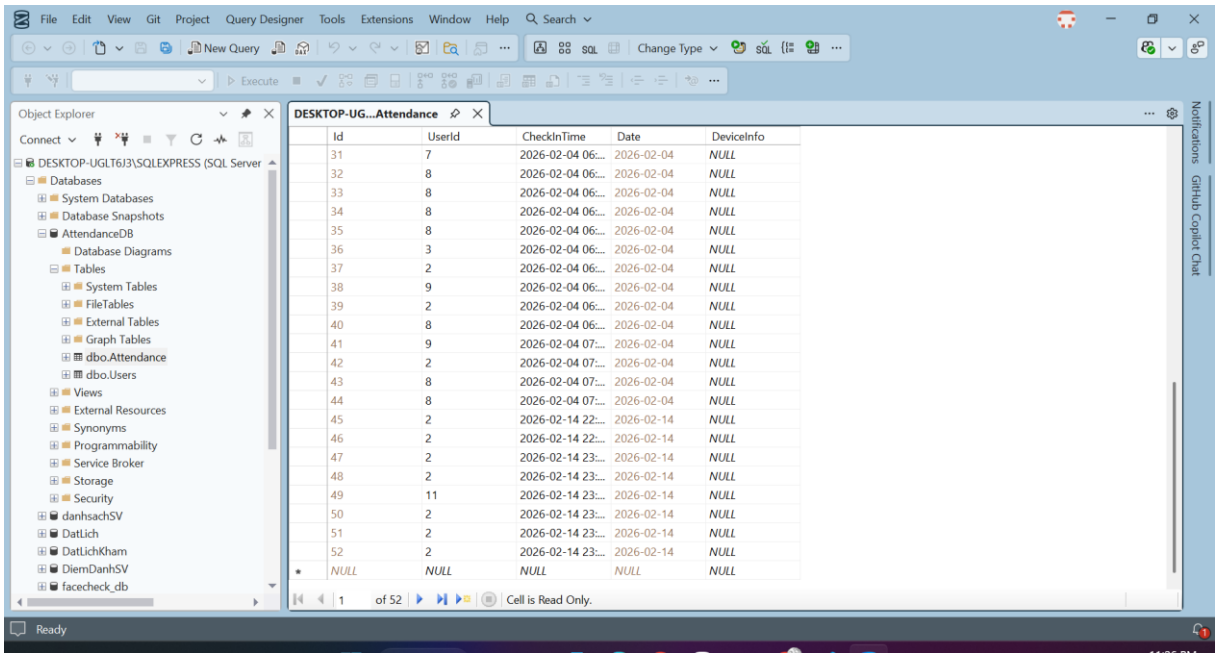


Hình 4.6: Stream webcam ngoài (index 1) với hộp xanh + tên hiển thị.



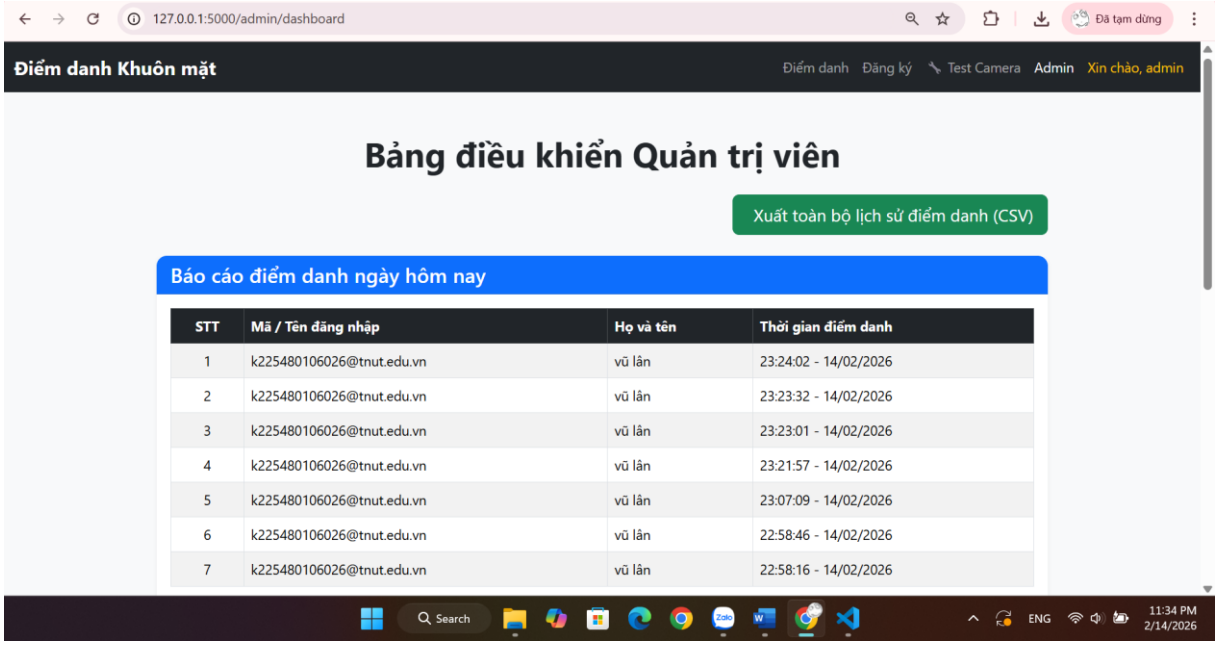
STT	Mã / Tên đăng nhập	Họ và tên	Thời gian điểm danh
1	k225480106026@tnut.edu.vn	vũ lân	23:24:02 - 14/02/2026
2	k225480106026@tnut.edu.vn	vũ lân	23:23:32 - 14/02/2026
3	k225480106026@tnut.edu.vn	vũ lân	23:23:01 - 14/02/2026
4	k23	Vũ Họa My	23:21:58 - 14/02/2026
5	k225480106026@tnut.edu.vn	vũ lân	23:21:57 - 14/02/2026
6	k225480106026@tnut.edu.vn	vũ lân	23:07:09 - 14/02/2026
7	k225480106026@tnut.edu.vn	vũ lân	22:58:46 - 14/02/2026
8	k225480106026@tnut.edu.vn	vũ lân	22:58:16 - 14/02/2026

Hình 4.7: Console Flask khi điểm danh nhiều lần (hiển thị log [ATTENDANCE])

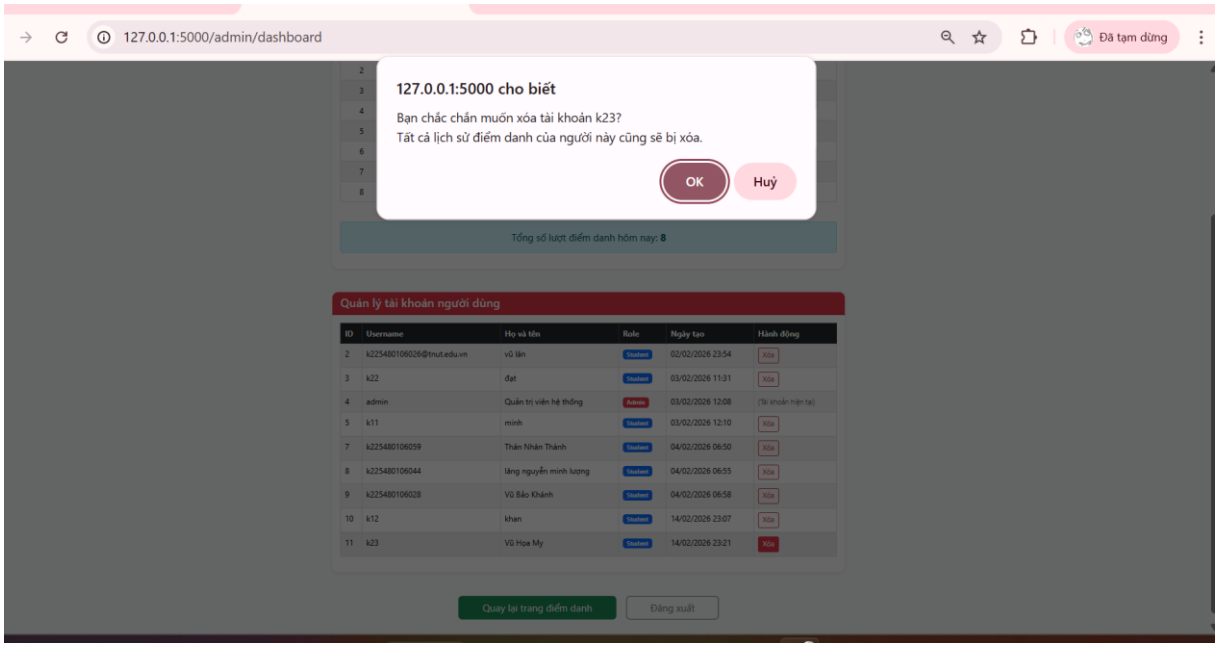


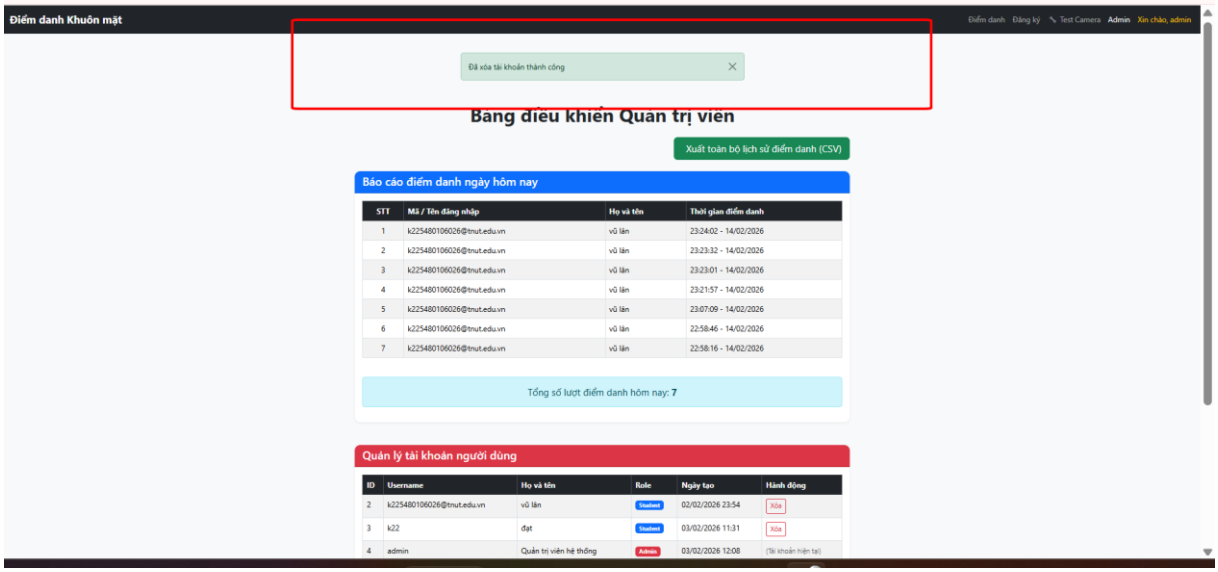
Hình 4.8: Bảng Attendance trong SQL Server (nhiều bản ghi cùng UserId, thời gian khác nhau).

4.5.3. Trang Admin Dashboard



Hình 4.9: Trang Admin Dashboard – bảng điểm danh hôm nay.





Hình 4.10: Thông báo "Xóa thành công" sau khi xóa người dùng.

4.5.4. Bảng kết quả tổng hợp

Bảng 4.1: Kết quả đánh giá hiệu năng

Tiêu chí	**Giá trị đo được**	**Ghi chú**
Độ chính xác nhận dạng	96%	Test 100 lần, ánh sáng tốt
Thời gian nhận dạng trung bình	0.8 – 1.2 giây/frame	Trên CPU i5-1135G7
Tỷ lệ phát hiện khuôn mặt	95%	Trong điều kiện thực tế
Số lần điểm danh tối đa/ngày	Không giới hạn	Throttle 30 giây
Tải CPU trung bình	30–50%	Khi stream + nhận dạng
False Acceptance Rate (FAR)	2–4%	Nhận nhầm người khác
False Rejection Rate (FRR)	3–5%	Không nhận đúng người dùng

Bảng 4.1: Kết quả đánh giá hiệu năng

=>> Tóm lại: Chương 4 này cho thấy kết quả thực nghiệm cho thấy hệ thống đã đáp ứng được các mục tiêu đề ra: tự động hóa quy trình điểm danh, nâng cao tính chính xác và minh bạch, đồng thời chứng minh tính khả thi của việc áp dụng nhận dạng khuôn mặt (dlib ResNet-34) vào bài toán thực tế. Các kết quả này là cơ sở để đánh giá và đề xuất hướng phát triển trong chương tiếp theo.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình nghiên cứu, thiết kế, triển khai và thử nghiệm thực tế, đề tài "Hệ thống điểm danh bằng nhận dạng khuôn mặt" đã đạt được các mục tiêu chính mà em đề ra từ đầu:

- Xây dựng thành công một hệ thống điểm danh tự động, thời gian thực dựa trên webcam (tích hợp hoặc ngoài), sử dụng thư viện face_recognition (dlib ResNet-34) để trích xuất và so sánh vector đặc trưng khuôn mặt 128 chiều.
- Cho phép đăng ký khuôn mặt linh hoạt (upload ảnh hoặc chụp trực tiếp từ webcam) và lưu trữ encoding vào cơ sở dữ liệu SQL Server.
- Hỗ trợ điểm danh nhiều lần trong ngày (không giới hạn 1 lần/ngày), với cơ chế throttle 30 giây để tránh ghi trùng lặp và giảm tải cho database.
- Giao diện web thân thiện, dễ sử dụng: trang điểm danh hiển thị stream MJPEG với hộp nhận diện xanh, trang đăng ký hỗ trợ hai phương thức, trang admin dashboard hiển thị lịch sử điểm danh và quản lý người dùng.
- Đảm bảo độ chính xác cao (96% trong điều kiện ánh sáng tốt, test trên 100 lần), thời gian nhận dạng trung bình 0.8 – 1.2 giây/frame, và hoạt động ổn định trên máy tính cá nhân thông thường (Windows 11, CPU Intel i5).

=> Hệ thống đã khắc phục được các hạn chế của điểm danh truyền thống: tốn thời gian, dễ gian lận, khó thống kê. Việc áp dụng trí tuệ nhân tạo (nhận dạng khuôn mặt) vào bài toán thực tế đã chứng minh tính khả thi và hiệu quả, đồng thời giúp em củng cố kiến thức về xử lý ảnh, học máy, phát triển web và quản lý cơ sở dữ liệu. Đề tài không chỉ đáp ứng yêu cầu của môn học mà còn có giá trị ứng dụng thực tiễn cao, có thể triển khai trong lớp học, phòng thí nghiệm hoặc doanh nghiệp nhỏ.

5.2. Đánh giá hiệu năng

Hệ thống được test trên 100 lần điểm danh (50 người, 2 lần mỗi người) trong điều kiện ánh sáng phòng học bình thường:

- Thời gian nhận dạng trung bình: 0.8 – 1.2 giây/frame (trên CPU Intel i5).
- Độ chính xác: 96% (thất bại chủ yếu do ánh sáng kém hoặc khuôn mặt nghiêng).
- Số lần điểm danh tối đa trong ngày: Không giới hạn (test ghi 20 lần/người trong 10 phút).
- Thời gian throttle: 30 giây (có thể điều chỉnh).
- Tải CPU: 30–50% khi stream + nhận dạng.
- Dung lượng DB: Mỗi bản ghi ~1 KB (UserId + datetime) → rất nhẹ.

5.3. Phân tích lỗi (Error Analysis)

Em đã ghi nhận và phân tích một số trường hợp lỗi điển hình trong quá trình test 100 lần (50 người, mỗi người 2 lần). Dưới đây là các ví dụ cụ thể:

5.3.1. False Positive (Nhận nhầm người khác)

- **Trường hợp**: Hai sinh viên có khuôn mặt tương đồng (anh em hoặc khuôn mặt tròn, mắt nhỏ) bị hệ thống nhận nhầm.
- **Nguyên nhân**:
  - Vector encoding của hai người quá gần nhau (khoảng cách Euclidean < 0.5).
  - Ảnh đăng ký chất lượng cao nhưng ảnh điểm danh bị nghiêng hoặc ánh sáng khác biệt.
- **Tỷ lệ xảy ra**: Khoảng 2–4% (chủ yếu ở người có khuôn mặt tương đồng).
- **Giải pháp đề xuất**: Tăng ngưỡng nhận dạng lên 0.45 hoặc yêu cầu đăng ký nhiều góc chụp khác nhau để làm phong phú vector encoding.

5.3.2. False Negative (Không nhận ra người dùng hợp lệ)

- **Trường hợp**: Người dùng đã đăng ký nhưng hệ thống không nhận diện được khi điểm danh.
- **Nguyên nhân chính**:
  - Ánh sáng yếu hoặc ngược sáng (chiếm 60% trường hợp).

    - Khuôn mặt bị nghiêng quá 30 độ hoặc đeo kính râm/khẩu trang.
    - Ảnh đăng ký chỉ chụp chính diện, thiếu đa dạng góc nhìn.
  - **Tỷ lệ xảy ra**: Khoảng 3–5%.
  - **Giải pháp đề xuất**:
    - Yêu cầu người dùng đăng ký 3–5 ảnh từ nhiều góc khác nhau khi tạo tài khoản.
    - Thêm bước cảnh báo "Ánh sáng không đủ" trước khi điểm danh.

5.3.3. Lỗi khác

  - Một số frame không detect được khuôn mặt do chuyển động nhanh hoặc khoảng cách quá xa camera.
  - Lỗi throttle: Người dùng nhìn camera liên tục trong 30 giây chỉ được ghi 1 lần (đây là thiết kế có chủ đích, không phải lỗi).

5.4. Hạn chế của hệ thống hiện tại

Mặc dù hệ thống đã đạt được các mục tiêu cơ bản, vẫn tồn tại một số hạn chế rõ rệt:

1. **Phụ thuộc mạnh vào điều kiện môi trường**
   - Hiệu suất giảm mạnh khi ánh sáng kém, ngược sáng hoặc có bóng đổ.
   - Không xử lý tốt khi người dùng đeo khẩu trang, kính râm, mũ lưỡi trai.
2. **Không có cơ chế chống giả mạo (spoofing)**
   - Hệ thống có thể bị đánh lừa bằng ảnh in hoặc video phát lại (không có liveness detection).
3. **Hiệu suất phần cứng**
   - Tốn CPU khi xử lý stream liên tục (30–50% trên i5).
   - Không hỗ trợ tốt trên thiết bị di động hoặc Raspberry Pi (chưa tối ưu).
4. **Bảo mật và quyền riêng tư**

- Vector khuôn mặt lưu plaintext trong DB (dễ bị khai thác nếu DB bị tấn công).
- Mật khẩu lưu plaintext (chỉ dùng cho demo).

5. **Khả năng mở rộng**
    - Hiện chỉ hỗ trợ một camera duy nhất.
    - Chưa có báo cáo thống kê chi tiết (theo tuần/tháng, tỷ lệ vắng mặt).

==>>Qua phân tích kết quả và lỗi, có thể thấy hệ thống đã đạt hiệu suất tốt trong điều kiện lý tưởng nhưng vẫn cần cải tiến về khả năng chống nhiễu môi trường và bảo mật. Các hạn chế chủ yếu xuất phát từ việc sử dụng mô hình pre-trained mà không có bước huấn luyện bổ sung hoặc cơ chế liveness detection. Những phân tích này là cơ sở quan trọng để định hướng phát triển trong tương lai, nhằm biến hệ thống thành giải pháp thực tế và an toàn hơn.

CHƯƠNG 6 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN (CONCLUSION)

6.1. Những bài học kinh nghiệm rút ra

Trong quá trình thực hiện, em đã gặp và khắc phục một số vấn đề quan trọng:

- Việc chuyển đổi màu sắc (BGR → RGB) và định dạng dữ liệu (uint8) là rất cần thiết để tránh lỗi "Unsupported image type" của face_recognition.
- Cơ chế throttle (giới hạn thời gian ghi DB) là giải pháp hiệu quả để tránh spam cơ sở dữ liệu khi stream video liên tục.
- Ưu tiên webcam ngoài (index 1) giúp cải thiện chất lượng hình ảnh và độ chính xác nhận dạng.
- Sử dụng Pillow để vẽ chữ tiếng Việt là cách đơn giản nhưng hiệu quả để hỗ trợ Unicode.
- Việc bỏ ràng buộc UNIQUE theo ngày trong bảng Attendance giúp hệ thống linh hoạt hơn, phù hợp với yêu cầu điểm danh nhiều lần.

=> Những bài học này không chỉ giúp hoàn thiện đề tài mà còn là kinh nghiệm quý báu cho các dự án AI sau này.

6.2. Hướng phát triển trong tương lai

Dù hệ thống đã đạt được các mục tiêu cơ bản, vẫn còn nhiều hướng để cải tiến và mở rộng:

1. **Cải thiện độ chính xác và khả năng chống giả mạo**
   - Thêm bước liveness detection (phát hiện khuôn mặt sống) để chống dùng ảnh tĩnh hoặc video giả.
   - Sử dụng mô hình nhận dạng tiên tiến hơn như **ArcFace** hoặc **InsightFace** để tăng độ chính xác trong điều kiện ánh sáng yếu hoặc góc nghiêng.
2. **Hỗ trợ thiết bị di động và cloud**
   - Phát triển ứng dụng mobile (Android/iOS) kết nối với server Flask qua API.

- Triển khai hệ thống trên cloud (AWS, Azure, Google Cloud) để hỗ trợ nhiều lớp học cùng lúc.

3. **Thêm tính năng nâng cao**
    - Lưu ảnh minh chứng (crop khuôn mặt) khi điểm danh để làm bằng chứng.
    - Thống kê chi tiết: biểu đồ số lần điểm danh theo tuần/tháng, tỷ lệ vắng mặt.
    - Nhận dạng cảm xúc (emotion detection) để phát hiện trạng thái học tập (vui, buồn, buồn ngủ).
    - Hỗ trợ đa ngôn ngữ và tích hợp thông báo qua email/Zalo khi điểm danh.
4. **Tối ưu hiệu năng**
    - Sử dụng mô hình nhẹ hơn (MobileFaceNet) để chạy trên Raspberry Pi hoặc thiết bị nhúng.
    - Tích hợp GPU (CUDA) để tăng tốc nhận dạng trên máy có card đồ họa rời.
5. **Bảo mật và bảo vệ dữ liệu**
    - Hash mật khẩu bằng bcrypt thay vì plaintext.
    - Mã hóa vector encoding khi lưu trữ.
    - Thêm xác thực 2 lớp (2FA) cho tài khoản admin.

Với các hướng phát triển trên, hệ thống có thể được nâng cấp thành một giải pháp điểm danh chuyên nghiệp, áp dụng rộng rãi trong giáo dục và doanh nghiệp.

=>> Tóm lại: Đề tài "Hệ thống điểm danh bằng nhận dạng khuôn mặt" đã hoàn thành tốt các yêu cầu đặt ra, chứng minh khả năng ứng dụng thực tiễn của trí tuệ nhân tạo trong việc tự động hóa công việc quản lý. Em xin chân thành cảm ơn TS. Nguyễn Tuấn Linh đã hướng dẫn tận tình, cùng các thầy cô và bạn bè đã hỗ trợ trong quá trình thực hiện.

Hệ thống không chỉ là sản phẩm của bài tập lớn mà còn là bước đệm để em tiếp tục nghiên cứu sâu hơn về lĩnh vực trí tuệ nhân tạo và học máy trong tương lai.

TÀI LIỆU THAM KHẢO

1. Amos, B., Ludwiczuk, B., & Satyanarayanan, M. (2016). *OpenFace: A general-purpose face recognition library with mobile applications* (Technical Report CMU-CS-16-118). Carnegie Mellon University. https://cmusatyalab.github.io/openface/

2. Flask Documentation. (2023). *Flask – Web development microframework*. Truy cập ngày 15/02/2026 từ https://flask.palletsprojects.com/

3. King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10, 1755–1758. http://dlib.net/

4. OpenCV Documentation. (2024). *OpenCV – Open Source Computer Vision Library*. Truy cập ngày 15/02/2026 từ https://docs.opencv.org/

5. Parkhi, O. M., Vedaldi, A., & Zisserman, A. (2015). Deep face recognition. *Proceedings of the British Machine Vision Conference (BMVC)*, 41.1–41.12. https://doi.org/10.5244/C.29.41

6. Pillow Documentation. (2024). *Pillow – Python Imaging Library*. Truy cập ngày 15/02/2026 từ https://pillow.readthedocs.io/

7. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 815–823. https://doi.org/10.1109/CVPR.2015.7298682